Fructus

Instruction Set Reference

2026-10-23 · f840fa3+ (uncommitted changes)

https://github.com/erikcorry/fructus

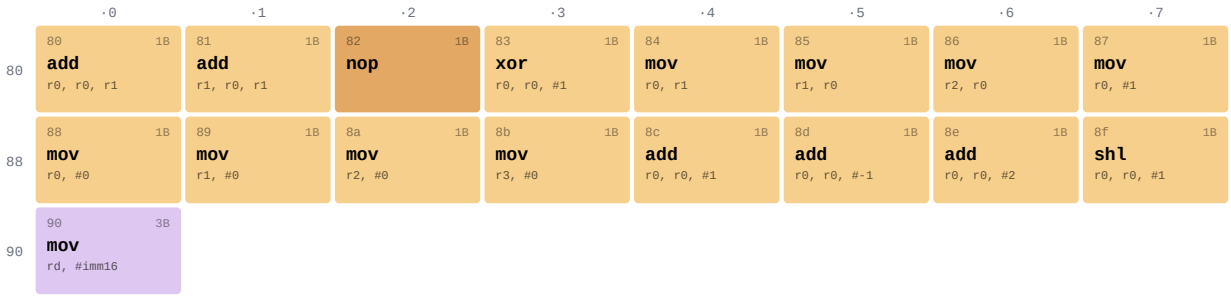
FRUCTUS · 16-BIT · FIRST-BYTE DECODE

Opcode Map

Instruction length comes from the first byte alone, so this table is the whole decoder.

169 ASSIGNED **87** FREE **16** ADDRESSING MODES **64** INSTRUCTION ENTRIES

	·0	·1	·2	·3	·4	·5	·6	·7
00	00 1B **brk**	01 1B **halt**	02 1B **jmp** r5	03 1B **call** r5	04 1B **ld** r0, [r0, #0]	05 1B **ld8** r0, [r0, #0]	06 1B **ld** r0, [r6, #0]	07 1B **st** r0, [r6, #0]
08	08 1B **ret**	09 1B **rti**	0a 1B **sei**	0b 1B **cli**	0c 1B **push** r7	0d 1B **pop** r7	0e 1B **push** r0	0f 1B **pop** r2
10	10	11 3B **st** rs, [ra, #off10]	12 2B **st** rs, [ra, #off3]	13 2B **st** rs, [ra, #off3]	14	15	16 2B **st** rs, [ra, rb]	17 2B **st** rs, [ra, rb]
18	18	19 3B **st8** rs, [ra, #off10]	1a 2B **st8** rs, [ra, #off3]	1b 2B **st8** rs, [ra, #off3]	1c	1d	1e 2B **st8** rs, [ra, rb]	1f 2B **st8** rs, [ra, rb]
20	20 2B **ld** rd, [rd, #off5]	21 3B **ld** rd, [ra, #off10]	22 2B **ld** rd, [ra, #off3]	23 2B **ld** rd, [ra, #off3]	24	25	26 2B **ld** rd, [ra, rb]	27 2B **ld** rd, [ra, rb]
28	28 2B **ld8** rd, [rd, #off5]	29 3B **ld8** rd, [ra, #off10]	2a 2B **ld8** rd, [ra, #off3]	2b 2B **ld8** rd, [ra, #off3]	2c	2d	2e 2B **ld8** rd, [ra, rb]	2f 2B **ld8** rd, [ra, rb]
30	30 2B **mov** rd, #imm5	31 3B **movhi** rd, ra, #imm10	32 2B **sxt8 bitrev** rd, ra, #op	33 2B **clz popcount** rd, ra, #op	34 2B **mov** rd, #immbit5	35 2B **mov** rd, #immask5	36 2B **movhi** rd, ra, rb	37 2B **movhi** rd, ra, rb
38	38 2B **add** rd, rd, #imm5	39 3B **add** rd, ra, #imm10	3a 2B **add** rd, ra, #imm3	3b 2B **add** rd, ra, #imm3	3c 2B **add** rd, rd, #immbit5	3d 2B **add** rd, rd, #immask5	3e 2B **add** rd, ra, rb	3f 2B **add** rd, ra, rb
40	40 2B **rsb** rd, rd, #imm5	41 3B **rsb** rd, ra, #imm10	42 2B **rsb** rd, ra, #imm3	43 2B **rsb** rd, ra, #imm3	44	45	46 2B **rsb** rd, ra, rb	47 2B **rsb** rd, ra, rb
48	48 2B **xor** rd, rd, #imm5	49 3B **xor** rd, ra, #imm10	4a 2B **xor** rd, ra, #imm3	4b 2B **xor** rd, ra, #imm3	4c 2B **xor** rd, rd, #immbit5	4d 2B **xor** rd, rd, #immask5	4e 2B **xor** rd, ra, rb	4f 2B **xor** rd, ra, rb
50	50 2B **or** rd, rd, #imm5	51 3B **or** rd, ra, #imm10	52 2B **or** rd, ra, #imm3	53 2B **or** rd, ra, #imm3	54 2B **or** rd, rd, #immbit5	55 2B **or** rd, rd, #immask5	56 2B **or** rd, ra, rb	57 2B **or** rd, ra, rb
58	58 2B **and** rd, rd, #imm5	59 3B **and** rd, ra, #imm10	5a 2B **and** rd, ra, #imm3	5b 2B **and** rd, ra, #imm3	5c 2B **and** rd, rd, #immbit5	5d 2B **and** rd, rd, #immask5	5e 2B **and** rd, ra, rb	5f 2B **and** rd, ra, rb
60	60 2B **shl** rd, rd, #imm5	61 3B **shl** rd, ra, #imm10	62 2B **shl** rd, ra, #imm3	63 2B **shl** rd, ra, #imm3	64	65	66 2B **shl** rd, ra, rb	67 2B **shl** rd, ra, rb
68	68 2B **asr** rd, rd, #imm5	69 3B **asr** rd, ra, #imm10	6a 2B **asr** rd, ra, #imm3	6b 2B **asr** rd, ra, #imm3	6c	6d	6e 2B **asr** rd, ra, rb	6f 2B **asr** rd, ra, rb
70	70 2B **lsr** rd, rd, #imm5	71 3B **lsr** rd, ra, #imm10	72 2B **lsr** rd, ra, #imm3	73 2B **lsr** rd, ra, #imm3	74	75	76 2B **lsr** rd, ra, rb	77 2B **lsr** rd, ra, rb
78	78 2B **iseq** rd, rd, #imm5	79 3B **iseq** rd, ra, #imm10	7a 2B **iseq** rd, ra, #imm3	7b 2B **iseq** rd, ra, #imm3	7c 2B **isset** rd, rd, #immbit5	7d 2B **isset** rd, rd, #immask5	7e 2B **iseq** rd, ra, rb	7f 2B **iseq** rd, ra, rb


·0
·1
·2
·3
·4
·5
·6
·7
80
80 1B add r0, r0, r1
81 1B add r1, r0, r1
82 1B nop
83 1B xor r0, r0, #1
84 1B mov r0, r1
85 1B mov r1, r0
86 1B mov r2, r0
87 1B mov r0, #1
88
88 1B mov r0, #0
89 1B mov r1, #0
8a 1B mov r2, #0
8b 1B mov r3, #0
8c 1B add r0, r0, #1
8d 1B add r0, r0, #-1
8e 1B add r0, r0, #2
8f 1B shl r0, r0, #1
90
90 3B mov rd, #imm16

Fructus Instruction Set Reference

Generated from isa/fructus.toml by tools/gen-manual.js. 16-bit registers r0–r7, a 16-bit address space, little-endian data, and instructions one to three bytes long.

Reading an Instruction Page

Section	Meaning
Opcode	The instruction's bytes in hex, byte 0 first. A range such as 3E–3F means the low bits of that byte belong to an operand field, so any value in the range is this instruction; xx is a byte made entirely of operand fields. The first byte alone decides the length of an instruction.
Instruction	The assembly syntax for that encoding. rd, ra, rb, rs and rc are any of r0–r7 (sp is r6 and lr is r7); a name such as r0 is a register the encoding pins. imm5, imm10, off8 and similar are signed fields of that many bits, sign-extended to 16 bits; immbit5, immask5, imm3 and shift3 are table fields whose values are listed on the page.
Bytes, Cycles	Length, and the cycles from this instruction's dispatch to the next one's, when the fetcher keeps up - see The cost model. "odd" is the cost when a word in memory is at an odd address; "taken" is a conditional branch that branches, guessed right; "wrong" is a branch guessed wrong, or a jump, call or ret through a register whose target was not guessed or guessed wrong. A taken branch, a jump, a call, a return, brk and rti cost one more when the instruction at their target reaches into a second word: two bytes at an odd address, or three at any - but not a call that went straight to its target, which holds dispatch while the second word comes in.
Form	The form's name in isa/fructus.toml, which also labels its diagram under Encodings. The assembler picks the shortest form that can hold the operands written.
Operation	The spec's semantics, which tools/sim.js executes. pc is already the address of the next instruction; R[n] is a register, M8[] and M16[] are memory (16-bit values little-endian); everything is truncated to 16 bits when stored.

Cycle Counts

The processor reads instructions ahead of itself: a fetcher brings in a word, two bytes, in every cycle the bus is free, into a buffer of three words, and an instruction dispatches once all its bytes are there. The count on each page is the instruction's own, from its dispatch to the next one's, when the next one's bytes are already in the buffer.

Usually they are. An ALU instruction of one or two bytes takes no more than the fetcher brings, and an instruction of two cycles or more - mul, a load, a store - lets it gain. The fetcher falls behind in a run of three-byte instructions, which take more than two bytes a cycle, and after a run of loads and stores, which take the bus. A taken branch, a jump, a call, a return, brk and rti empty the buffer and read their target, and their count includes that, the target arriving with nothing to spare. On the benchmark kernels in tests/bench, 6% of all cycles are spent waiting for the fetcher; tools/sim.js models it, and its counts are exact.

Built in blit mode, for a system whose data can be at the far side of the chip, ld, ld8 and ldm take a cycle more.

Code may not write just ahead of itself. A store to the six bytes after the instruction being dispatched reaches memory but not the fetcher's copy. Code that writes instructions must reach them through a taken branch, a jump, a call, a return or rti, each of which empties the buffer and reads memory afresh.

Contents

	Opcode map	2
add	rd = ra + rb; rd = ra + immediate	6
and	rd = ra & rb; rd = ra & immediate	9
asr	rd = ra >> (rb & 15), sign propagating; rd = ra >> (imm & 15), sign propagating	11
bitrev	rd = ra with its 16 bits reversed	13
br	compare two registers and branch; compare against a table constant and branch	14
brclr	branch if ra & mask is zero	18
brk	software interrupt: enter the handler	20
brset	branch if ra & mask is non-zero	21
call	lr = pc of next instruction; pc = imm16; lr = pc of next instruction; pc = ra	23
cli	disable interrupts	25
clz	rd = count of leading zeros in ra	26

halt	stop the machine until an interrupt	27
iseq	rd = 1 if ra equals rb, else 0; rd = 1 if ra equals a constant, else 0	28
ishi	rd = 1 if ra is higher than rb, unsigned, else 0	30
islo	rd = 1 if ra is lower than rb, unsigned, else 0 (alias)	31
isset	rd = 1 if ra and the mask share a bit, else 0	32
jmp	pc = imm16; pc = ra	33
ld	load 16-bit; load 16-bit through a register index	35
ld8	load 8-bit, zero extended; load 8-bit through a register index, zero extended	37
ldm	load ra then rb through r1; load ra, then rb, then rc through r1	39
lsr	rd = ra >> (rb & 15), zero filling; rd = ra >> (imm & 15), zero filling	40
mov	rd = immediate; rd = ra	42
movhi	rd = (rb << 8) | (ra & 0xff); rd = (immediate << 8) | (ra & 0xff)	45
mul	rd = low 16 bits of ra * rb; rd = low 16 bits of ra * immediate	46
mulhs	rd = high 16 bits of ra * rb, signed	47
mulhu	rd = high 16 bits of ra * rb, unsigned	48
nop	do nothing	49
or	rd = ra | rb; rd = ra | immediate	50
pop	pop a word into ra; pop two words, into ra then rb; pop three words, into ra then rb then rc	53
popcount	rd = number of set bits in ra	55
push	push ra; push ra then rb; push ra, then rb, then rc	56
ret	pc = lr	58
rsb	rd = rb - ra (reverse subtract); rd = immediate - ra	59
rti	return from an interrupt	61
sei	enable interrupts	62
shl	rd = ra << (rb & 15); rd = ra << (imm & 15)	63
st	store 16 bits; store 16 bits through a register index	65
st8	store low 8 bits; store low 8 bits through a register index	67
stm	store ra then rb through r5; store ra, then rb, then rc through r5	69
sub	rd = ra - rb; rd = ra - immediate (alias)	71
sxt8	rd = ra sign extended from 8 bits	72
xor	rd = ra ^ rb; rd = ra ^ immediate	73

ADD — rd = ra + rb; rd = ra + immediate

Opcode	Instruction	Bytes	Cycles	Form	Operands
80	add r0, r0, r1	1	1	r0_r0_r1	All operands implicit
81	add r1, r0, r1	1	1	r1_r0_r1	All operands implicit
3E–3F xx	add rd, ra, rb	2	1	three_reg	
8C	add r0, r0, #1	1	1	inc_r0	All operands implicit
8D	add r0, r0, #-1	1	1	dec_r0	All operands implicit
8E	add r0, r0, #2	1	1	inc2_r0	All operands implicit
38 xx	add rd, rd, #imm5	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
3C xx	add rd, rd, #immbit5	2	1	reg_immbit5	immbit5: one of the 32 values below; ra is rd
3D xx	add rd, rd, #immask5	2	1	reg_immask5	immask5: one of the 32 values below; ra is rd
3A–3B xx	add rd, ra, #imm3	2	1	reg_reg_imm3	imm3: one of the 8 values below
39 xx xx	add rd, ra, #imm10	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

r0_r0_r1 **80** `add r0, r0, r1`

byte 0: 80

0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	1

Two fully implicit one-byte encodings for the two register triples that dominate in practice, r0 = r0 + r1 and r1 = r1 + r0. The second is written with its sources swapped, r0 then r1 as in the first, so that the two opcodes differ in the destination bit alone; a compiler puts a commutative pair in that order. Listed first only for readability - forms are tried shortest-first regardless of the order they appear here.

r1_r0_r1 **81** `add r1, r0, r1`

byte 0: 81

0	1	2	3	4	5	6	7
1	0	0	0	0	0	0	1

three_reg **3E–3F xx** `add rd, ra, rb`

byte 0: 3E–3F

0	1	2	3	4	5	6	7
b0	1	1	1	1	1	0	0
rb[0]							

byte 1: xx

0	1	2	3	4	5	6	7
d0	d1	d2	a0	a1	a2	b1	b2
rd			ra			rb[2:1]	

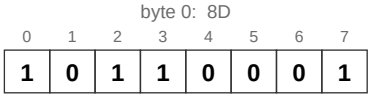
inc_r0 **8C** `add r0, r0, #1`

byte 0: 8C

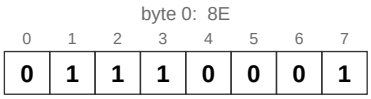
0	1	2	3	4	5	6	7
0	0	1	1	0	0	0	1

Increment, decrement and step-by-two on r0, in one byte each. Everything is pinned, so these carry no fields at all.

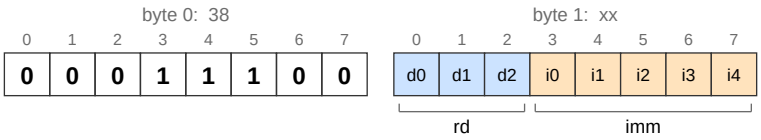
dec_r0 **8D** `add r0, r0, #-1`



inc2_r0 **8E** `add r0, r0, #2`

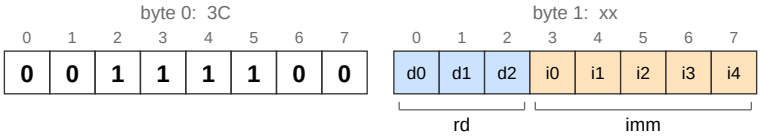


reg_imm5 **38 xx** `add rd, rd, #imm5`



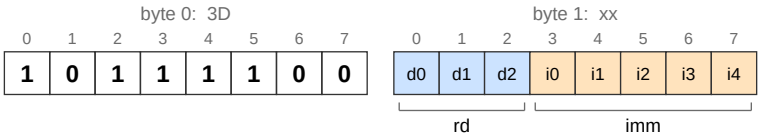
Destination doubles as the first source, which frees five bits for the immediate. Prints as ‘add r3, r3, #20‘.

reg_immbit5 **3C xx** `add rd, rd, #immbit5`



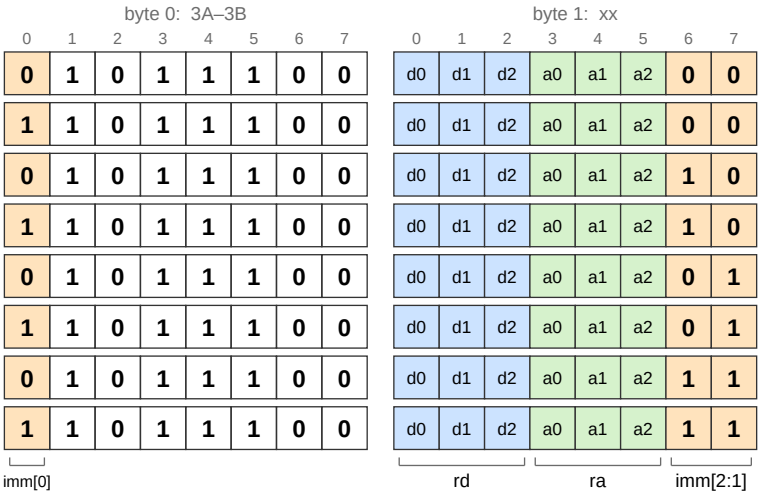
ADDING A POWER OF TWO. imm5 already reaches 1, 2, 4 and 8, so what this buys is 16 upwards and 0x8000 - and the complements, which are the subtractions: ‘add rd, rd, #0xfff0‘ is minus sixteen. The slot was reserved for immbit5 in every one of the eight ALU groups and nothing had asked for it here.

reg_immask5 **3D xx** `add rd, rd, #immask5`



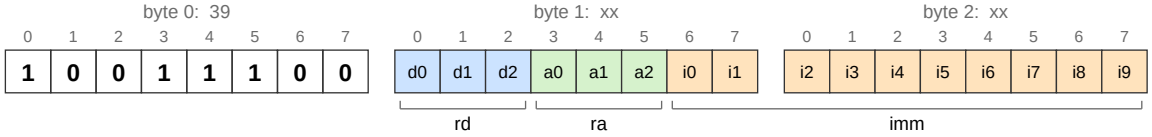
And the mask table, for the same reason it is on and/or/xor: a field is often added to as well as masked out.

reg_reg_imm3 **3A–3B xx** `add rd, ra, #imm3`



3A 00–3F	`add rd, ra, #-1`	
3B 00–3F	`add rd, ra, #0`	
3A 40–7F	`add rd, ra, #1`	
3B 40–7F	`add rd, ra, #2`	
3A 80–BF	`add rd, ra, #3`	
3B 80–BF	`add rd, ra, #4`	
3A C0–FF	`add rd, ra, #6`	
3B C0–FF	`add rd, ra, #8`	

reg_reg_imm10 **39 xx xx** `add rd, ra, #imm10`



Operand Values

immbit5: value by field value

0	0x0001	1	0x0002	2	0x0004	3	0x0008	4	0x0010	5	0x0020	6	0x0040	7	0x0080
8	0x0100	9	0x0200	10	0x0400	11	0x0800	12	0x1000	13	0x2000	14	0x4000	15	0x8000
16	0xFFFE	17	0xFFFD	18	0xFFFB	19	0xFFF7	20	0xFFEF	21	0xFFDF	22	0xFFBF	23	0xFF7F
24	0xFEFF	25	0xFDFF	26	0xFBFF	27	0xF7FF	28	0xEFFF	29	0xDFFF	30	0xBFFF	31	0x7FFF

immask5: value by field value

0	0x000F	1	0x0003	2	0xFF00	3	0x000C	4	0x00F0	5	0x0030	6	0xF0F0	7	0x00C0
8	0x0F00	9	0x0300	10	0xCCCC	11	0x0C00	12	0xF000	13	0x3000	14	0xAAAA	15	0xC000
16	0xFFF0	17	0xFFFC	18	0x00FF	19	0xFFF3	20	0xFF0F	21	0xFFCF	22	0x0F0F	23	0xFF3F
24	0xF0FF	25	0xFCFF	26	0x3333	27	0xF3FF	28	0x0FFF	29	0xCFFF	30	0x5555	31	0x3FFF

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

add rd, ra, rb

```
R[d] = R[a] + R[b];
```

add rd, ra, #imm

```
R[d] = R[a] + imm;
```

Notes

Six opcodes, four forms, split across two entries because the second source is a register in one shape and an immediate in the others.

All ALU operations are 16-bit only; there is no width bit in this block and no 8-bit add.

Entry 1 of 2: the three-register form. It buys its third register the same way the imm3 forms buy their bottom immediate bit - by stealing the low bit of the opcode byte, so it occupies an adjacent pair.

Entry 2 of 2: the immediate forms. These are exactly the memory block's 000 / 01i / 100 shapes, reused unchanged - a tied register with a wide 5-bit immediate, two registers with a table immediate, two registers with imm10.

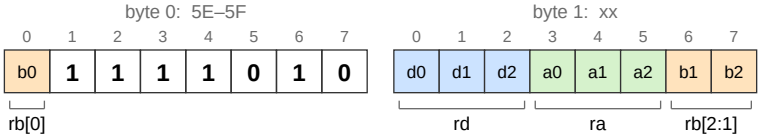
AND — rd = ra & rb; rd = ra & immediate

Opcode	Instruction	Bytes	Cycles	Form	Operands
`5E–5F xx`	`and rd, ra, rb`	2	1	three_reg	
`58 xx`	`and rd, rd, #imm5`	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
`5C xx`	`and rd, rd, #immbit5`	2	1	reg_immbit5	immbit5: one of the 32 values below; ra is rd
`5D xx`	`and rd, rd, #immask5`	2	1	reg_immask5	immask5: one of the 32 values below; ra is rd
`5A–5B xx`	`and rd, ra, #imm3`	2	1	reg_reg_imm3	imm3: one of the 8 values below
`59 xx xx`	`and rd, ra, #imm10`	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

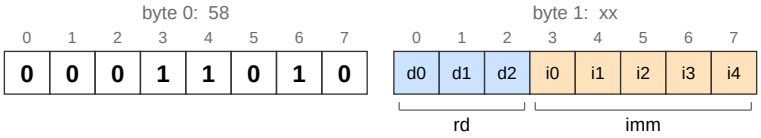
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

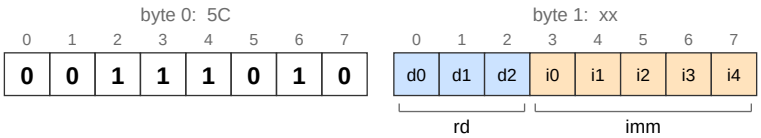
three_reg **5E–5F xx** `and rd, ra, rb`



reg_imm5 **58 xx** `and rd, rd, #imm5`

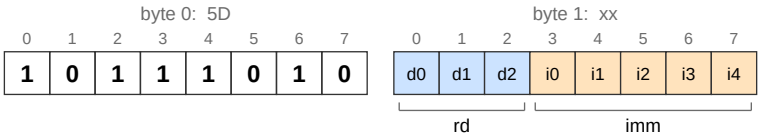


reg_immbit5 **5C xx** `and rd, rd, #immbit5`



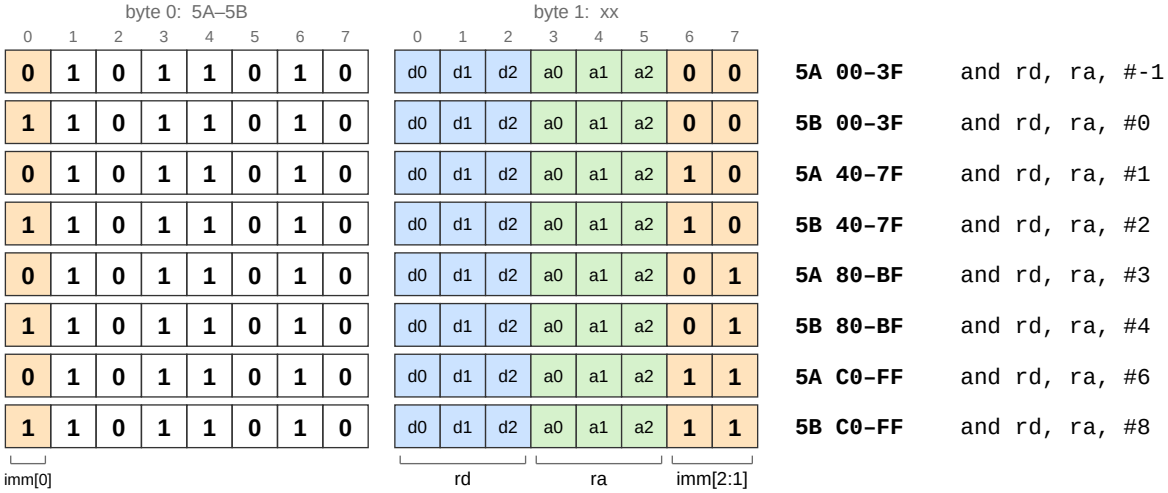
BIT CLEAR. ‘and rd, rd, #~(1<<n)‘ for any n, in two bytes. This is the half of immbit5 that would have gone to waste if the fifth bit had stayed a set/clear selector, and it is the more useful half: masking the sign bit off a value is one instruction now instead of a shl/lsr pair.

reg_immask5 **5D xx** `and rd, rd, #immask5`



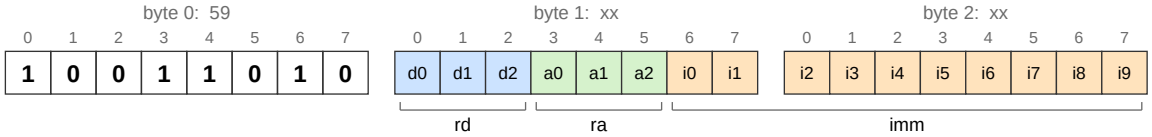
The same shape again, read through immask5: fields and stripes rather than single bits. Column .5 of the ALU block was reserved and empty.

reg_reg_imm3 **5A–5B xx** `and rd, ra, #imm3`



5A 00–3F	`and rd, ra, #-1`
5B 00–3F	`and rd, ra, #0`
5A 40–7F	`and rd, ra, #1`
5B 40–7F	`and rd, ra, #2`
5A 80–BF	`and rd, ra, #3`
5B 80–BF	`and rd, ra, #4`
5A C0–FF	`and rd, ra, #6`
5B C0–FF	`and rd, ra, #8`

reg_reg_imm10 **59 xx xx** `and rd, ra, #imm10`



Operand Values

immbit5: value by field value

0	0x0001	1	0x0002	2	0x0004	3	0x0008	4	0x0010	5	0x0020	6	0x0040	7	0x0080
8	0x0100	9	0x0200	10	0x0400	11	0x0800	12	0x1000	13	0x2000	14	0x4000	15	0x8000
16	0xFFFE	17	0xFFFD	18	0xFFFB	19	0xFFF7	20	0xFFEF	21	0xFFDF	22	0xFFBF	23	0xFF7F
24	0xFEFF	25	0xFDFF	26	0xFBFF	27	0xF7FF	28	0xEFFF	29	0xDFFF	30	0xBFFF	31	0x7FFF

immask5: value by field value

0	0x000F	1	0x0003	2	0xFF00	3	0x000C	4	0x00F0	5	0x0030	6	0xF0F0	7	0x00C0
8	0x0F00	9	0x0300	10	0xCCCC	11	0x0C00	12	0xF000	13	0x3000	14	0xAAAA	15	0xC000
16	0xFFF0	17	0xFFFC	18	0x00FF	19	0xFFF3	20	0xFF0F	21	0xFFCF	22	0x0F0F	23	0xFF3F
24	0xF0FF	25	0xFCFF	26	0x3333	27	0xF3FF	28	0x0FFF	29	0xCFFF	30	0x5555	31	0x3FFF

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

and rd, ra, rb

```
R[d] = R[a] & R[b];
```

and rd, ra, #imm

```
R[d] = R[a] & imm;
```

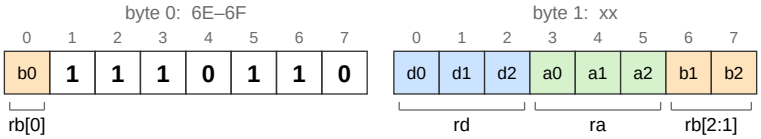
ASR — rd = ra >> (rb & 15), sign propagating; rd = ra >> (imm & 15), sign propagating

Opcode	Instruction	Bytes	Cycles	Form	Operands
6E–6F xx	asr rd, ra, rb	2	1	three_reg	
68 00–7F	asr rd, rd, #imm5	2	1	reg_imm5	imm5: 0 to 15; bit 4 ignored, written 0; ra is rd
6A–6B xx	asr rd, ra, #shift3	2	1	reg_reg_imm3	shift3: one of the 8 values below
69 xx 00–03	asr rd, ra, #imm10	3	1	reg_reg_imm10	imm10: 0 to 15; bits 9:4 ignored, written 0

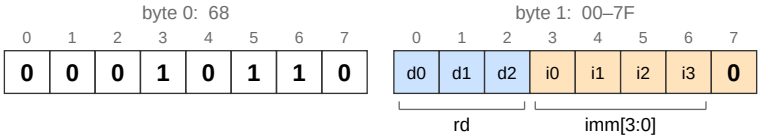
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

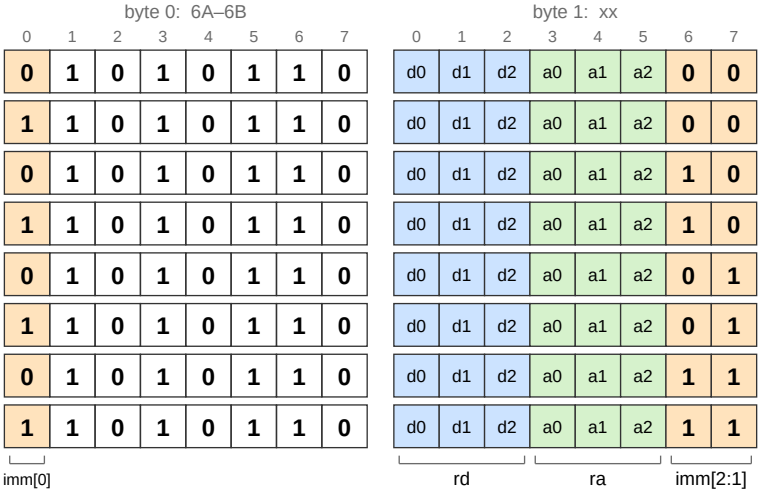
three_reg **6E–6F xx** asr rd, ra, rb



reg_imm5 **68 00–7F** asr rd, rd, #imm5

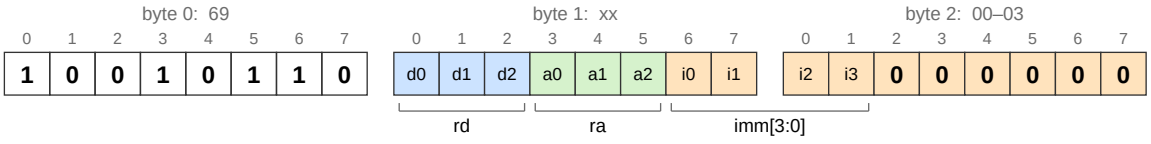


reg_reg_imm3 **6A–6B xx** asr rd, ra, #shift3



6A 00–3F	asr rd, ra, #15	
6B 00–3F	asr rd, ra, #0	
6A 40–7F	asr rd, ra, #1	
6B 40–7F	asr rd, ra, #2	
6A 80–BF	asr rd, ra, #3	
6B 80–BF	asr rd, ra, #4	
6A C0–FF	asr rd, ra, #6	
6B C0–FF	asr rd, ra, #8	

reg_reg_imm10 **69 xx 00–03** asr rd, ra, #imm10



Operand Values

shift3: value by field value

0 15 1 0 2 1 3 2 4 3 5 4 6 6 7 8

Operation

asr rd, ra, rb

```
R[d] = asr(R[a], R[b] & 15);
```

asr rd, ra, #imm

```
R[d] = asr(R[a], imm & 15);
```

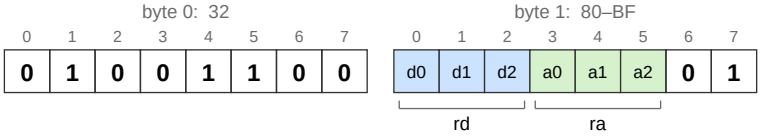
BITREV — rd = ra with its 16 bits reversed

Opcode	Instruction	Bytes	Cycles	Form	Operands
32 80–BF	bitrev rd, ra	2	1	reg_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **32 80–BF** bitrev rd, ra



Operation

```
R[d] = bitrev(R[a]);
```

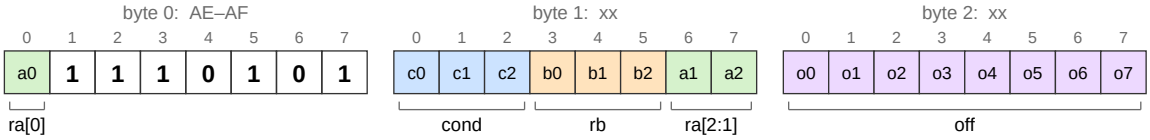
BR — compare two registers and branch; compare against a table constant and branch

Opcode	Instruction	Bytes	Cycles	Form	Operands
`AE–AF xx xx`	`br cond, ra, rb, off8`	3	1, taken 3, wrong 5	cond_reg_reg	off8: -128 to 127, sign-extended to 16 bits
`A8 xx xx`	`br cond, ra, #imm, off8`	3	1, taken 3, wrong 5	cond_reg_imm	(cond, imm): one of the 30 pairs below; off8: -128 to 127, sign-extended to 16 bits

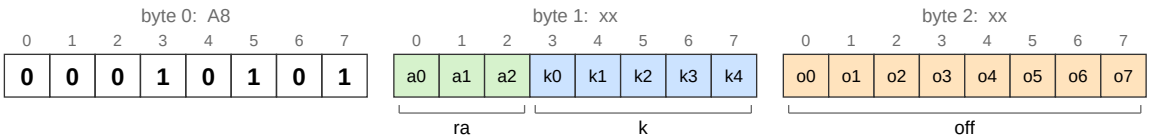
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

cond_reg_reg **AE–AF xx xx** `br cond, ra, rb, off8`



cond_reg_imm **A8 xx xx** `br cond, ra, #imm, off8`



Operand Values

cond3: condition by field value

`0` `lt` `1` `lo` `2` `le` `3` `ls` `4` `eq` `5` `ne` `6` `vs` `7` `vc`

Also accepted: cc is lo; gt is lt with the two operands exchanged; ge is le with the two operands exchanged; hi is lo with the two operands exchanged; hs is ls with the two operands exchanged; cs is ls with the two operands exchanged.

condimm5: (cond, imm) by field value

`0` `eq 0`	`1` `ne 0`	`2` `eq 4`	`3` `ne 4`	`4` `eq 1`	`5` `ne 1`	`6` `lt 1`	`7` `ge 1`
`8` `eq -1`	`9` `ne -1`	`10` `lt -1`	`11` `ge -1`	`12` `eq 2`	`13` `ne 2`	`14` `lt 2`	`15` `ge 2`
`16` `lt 4`	`17` `ge 8`	`18` `ge 32767`	`19` `ge 32766`	`20` `lt -32767`	`21` `lt -32766`	`22` `lo 10`	`23` `hs 10`
`24` `hs 3`	`25` `lo 32`	`26` `hs 32`	`27` `lo 6`	`28` `lo 8`	`29` `hs 8`		

Field values 30–31 are free: no instruction encodes them.

Conditions

A condition compares the first operand, a, with the second, b or the constant, in that order: lt means a < b. The letters are ARM's: signed comparisons say Less and Greater, unsigned ones LOwer and HIgher.

Name	Branches when
`eq`	EQual: the two values are the same bit pattern. Signed and unsigned agree.
`ne`	Not Equal: the two values differ in any bit.
`lt`	Less Than, signed: a < b reading both as two's complement, -32768 to 32767. -1 is less than 0.
`le`	Less than or Equal, signed: a <= b as two's complement.
`gt`	Greater Than, signed: a > b as two's complement. Encoded as lt with the operands exchanged.
`ge`	Greater than or Equal, signed: a >= b as two's complement. Encoded as le with the operands exchanged.
`lo`	LOwer, unsigned: a < b reading both as 0 to 65535. 0xffff is the largest value, not -1.
`ls`	Lower or Same, unsigned: a <= b as 0 to 65535.
`hi`	HIgher, unsigned: a > b as 0 to 65535. Encoded as lo with the operands exchanged.

Name	Branches when
hs	Higher or Same, unsigned: a >= b as 0 to 65535. Encoded as ls with the operands exchanged.
cc	Carry Clear: another name for lo, from ARM, where a - b clears the carry flag exactly when it borrows - when a is lower. Fructus has no flags; only the name is kept.
cs	Carry Set: another name for hs, from ARM - a - b does not borrow.
vs	oVerflow Set: a - b does not fit in a signed 16-bit result - the true difference is above 32767 or below -32768. Registers only: there is no vs against a constant, and none is needed, since the overflow tests that matter are plain comparisons (see condimm5).
vc	oVerflow Clear: a - b fits in a signed 16-bit result. Registers only.

Spellings of the Constant Comparisons

Each entry of condimm5 is a test on the register, and most tests can be written more than one way. The assembler accepts any of them and encodes the entry; a disassembler prints the entry's own spelling.

Field	Entry	Also written
0	`eq #0`	`lo #1, ls #0`
1	`ne #0`	`hi #0, hs #1`
2	`eq #4`	—
3	`ne #4`	—
4	`eq #1`	—
5	`ne #1`	—
6	`lt #1`	`le #0`
7	`ge #1`	`gt #0`
8	`eq #-1`	`hi #65534, hs #65535`
9	`ne #-1`	`lo #65535, ls #65534`
10	`lt #-1`	`le #-2`
11	`ge #-1`	`gt #-2`
12	`eq #2`	—
13	`ne #2`	—
14	`lt #2`	`le #1`
15	`ge #2`	`gt #1`
16	`lt #4`	`le #3`
17	`ge #8`	`gt #7`
18	`ge #32767`	`eq #32767, gt #32766`
19	`ge #32766`	`gt #32765`
20	`lt #-32767`	`le #-32768, eq #-32768`
21	`lt #-32766`	`le #-32767`
22	`lo #10`	`ls #9`
23	`hs #10`	`hi #9`
24	`hs #3`	`hi #2`
25	`lo #32`	`ls #31`
26	`hs #32`	`hi #31`
27	`lo #6`	`ls #5`
28	`lo #8`	`ls #7`
29	`hs #8`	`hi #7`

Same Test, Other Instruction

Some comparisons against a constant are bit tests: R[a] <u 256 holds exactly when none of bits 8–15 is set, and R[a] < 0 when bit 15 is. When the test written has no encoding of its own but the other instruction encodes the same test, the assembler uses that instead, in either direction - same length, same cycles, same branch. A disassembler prints what was encoded.

Written	Assembles as
`br lt, ra, #0`	`brset ra, #0x8000`
`br le, ra, #-1`	`brset ra, #0x8000`
`br hi, ra, #32767`	`brset ra, #0x8000`
`br hs, ra, #32768`	`brset ra, #0x8000`
`br lo, ra, #2`	`brclr ra, #0xFFFE`
`br ls, ra, #1`	`brclr ra, #0xFFFE`
`br lo, ra, #4`	`brclr ra, #0xFFFC`
`br ls, ra, #3`	`brclr ra, #0xFFFC`
`br lo, ra, #16`	`brclr ra, #0xFFF0`
`br ls, ra, #15`	`brclr ra, #0xFFF0`
`br lo, ra, #256`	`brclr ra, #0xFF00`
`br ls, ra, #255`	`brclr ra, #0xFF00`
`br lo, ra, #4096`	`brclr ra, #0xF000`
`br ls, ra, #4095`	`brclr ra, #0xF000`
`br lo, ra, #16384`	`brclr ra, #0xC000`
`br ls, ra, #16383`	`brclr ra, #0xC000`
`br lo, ra, #32768`	`brclr ra, #0x8000`
`br ls, ra, #32767`	`brclr ra, #0x8000`
`br gt, ra, #-1`	`brclr ra, #0x8000`
`br ge, ra, #0`	`brclr ra, #0x8000`
`br hi, ra, #1`	`brset ra, #0xFFFE`
`br hs, ra, #2`	`brset ra, #0xFFFE`
`br hi, ra, #3`	`brset ra, #0xFFFC`
`br hs, ra, #4`	`brset ra, #0xFFFC`
`br hi, ra, #15`	`brset ra, #0xFFF0`
`br hs, ra, #16`	`brset ra, #0xFFF0`
`br hi, ra, #255`	`brset ra, #0xFF00`
`br hs, ra, #256`	`brset ra, #0xFF00`
`br hi, ra, #4095`	`brset ra, #0xF000`
`br hs, ra, #4096`	`brset ra, #0xF000`
`br hi, ra, #16383`	`brset ra, #0xC000`
`br hs, ra, #16384`	`brset ra, #0xC000`
`brclr ra, #0xFFFF`	`br eq, ra, #0`
`brclr ra, #0xFFE0`	`br lo, ra, #32`
`brclr ra, #0xFFF8`	`br lo, ra, #8`
`brset ra, #0xFFFF`	`br ne, ra, #0`
`brset ra, #0xFFE0`	`br hs, ra, #32`
`brset ra, #0xFFF8`	`br hs, ra, #8`

Operation

br cond, ra, rb, off

```
if (test(cond, R[a], R[b], 16)) pc = pc + off;
```

A condition the encoding lacks is written by exchanging a and b; see the condition table.

br cond, ra, #imm, off

```
if (test(k.cond, R[a], k.imm, 16)) pc = pc + off;
```

Notes

Compare and branch. ‘br cond, ra, rb, L‘ branches to L when R[a] cond R[b] holds, and ‘br cond, ra, #k, L‘ when R[a] cond k does; otherwise execution falls through to the next instruction. L must lie within -128..+127 bytes of the next instruction. Nothing else changes: there are no flags, so a comparison and the branch that uses it are always one instruction.

The conditions are two letters, eq, ne, lt, le, gt, ge, lo, ls, hi, hs, vs and vc, plus cc and cs for lo and hs, and are explained one by one under Conditions. Every one is available between two registers. Against a constant, the pair (condition, constant) must be one of the 30 the condimm5 table holds - a constant costs no extra byte because the pair is one 5-bit index - though each pair can be written in all of its spellings: ‘br le, ra, #3‘ is the entry ‘lt #4‘, and ‘br eq, ra, #32767‘ is ‘ge #32767‘, since nothing is above 32767. A comparison the table lacks may still be one instruction, because some comparisons are bit tests: ‘x <u 256‘ asks whether any of bits 8-15 is set, and ‘x < 0‘ is the sign bit. The assembler writes those as ‘brclr‘ or ‘brset‘, in the same three bytes - see Same Test, Other Instruction. Anything else needs the constant in a register first.

HOW IT WORKS. The comparison is a subtract whose result is discarded; only the condition it sets is used, at 16-bit width.

The two-register form is the ‘rsb‘ three-register encoding with its DESTINATION field replaced by a 3-bit condition, plus a displacement byte:

THE TWO REGISTERS ARE THE OTHER WAY ROUND FROM THE SYNTAX, and that is what lets the comparison be an ‘rsb‘ with no extra multiplexing. ‘rsb rd, ra, rb‘ computes rb - ra: the SPLIT field minus the byte1[5:3] field. ‘br cond, ra, rb‘ tests the flags of R[a] - R[b]. For those to be the same subtraction, the first register NAMED has to sit in the split field, so ‘a‘ takes z and ee while ‘b‘ takes byte1[5:3].

Read as datapath rather than as syntax there is no reversal at all:

```
byte1[5:3]              always ALU port A
{byte1[7:6], opcode[0]} always ALU port B
```

and every three-operand instruction in the map obeys it - the ALU triples, the shift triples, push and pop’s triples, and now both branches. rtl/immgen.sv emits port B’s register number at +6 and +7 for all of them.

Three of the register bits and all of the register decode are therefore shared with the ALU three-register forms; only byte 1’s top field changes meaning. Eight encoded conditions cover more than eight spellings because the two registers can be exchanged - see ‘cond3‘.

The single-register form replaces both the second register and the condition with one 5-bit index into the ‘condimm5‘ table, which supplies a condition and a constant together:

BYTE 1 IS THE MASK BRANCHES’ BYTE 1. ‘aaak_kkkk‘ here and ‘aaam_mmmm‘ in brclr and brset are one register and five bits in the same places; only the table reading those five bits differs. So these three share a row the way an ALU group’s +0, +4 and +5 forms do, and for the same reason - see the opcode map at the top of this file.

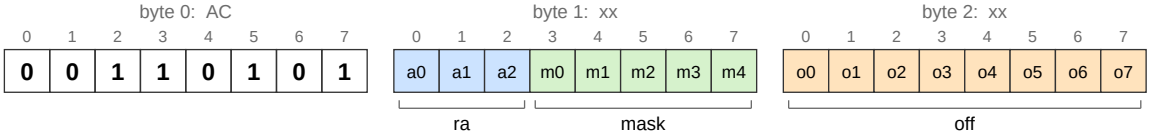
BRCLR — branch if ra & mask is zero

Opcode	Instruction	Bytes	Cycles	Form	Operands
`AC xx xx`	`brclr ra, #immbit5, off8`	3	1, taken 3, wrong 5	reg_mask	immbit5: one of the 32 values below; off8: -128 to 127, sign-extended to 16 bits
`AD xx xx`	`brclr ra, #immask5, off8`	3	1, taken 3, wrong 5	reg_mask5	immask5: one of the 32 values below; off8: -128 to 127, sign-extended to 16 bits

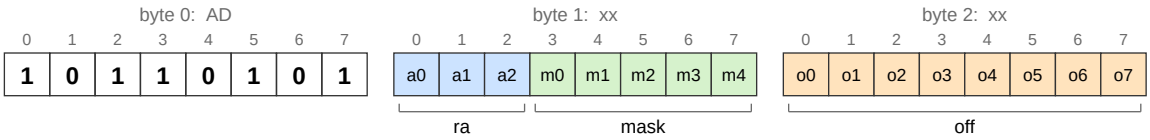
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_mask **`AC xx xx`** `brclr ra, #immbit5, off8`



reg_mask5 **`AD xx xx`** `brclr ra, #immask5, off8`



The same test against the mask table instead of a single bit - branch on whether a whole FIELD is empty. Same byte 1, same 5 bits, different table: +4 of the row reads them as immbit5 and +5 reads them as immask5, which is the relationship every ALU group has between its two mask forms. Clear and set are separated by BIT 3 rather than bit 0 - brclr is the upper row and brset the row eight below - so both spellings of each land in one column.

Operand Values

immbit5: value by field value

0 `0x0001`	1 `0x0002`	2 `0x0004`	3 `0x0008`	4 `0x0010`	5 `0x0020`	6 `0x0040`	7 `0x0080`
8 `0x0100`	9 `0x0200`	10 `0x0400`	11 `0x0800`	12 `0x1000`	13 `0x2000`	14 `0x4000`	15 `0x8000`
16 `0xFFFE`	17 `0xFFFD`	18 `0xFFFB`	19 `0xFFF7`	20 `0xFFEF`	21 `0xFFDF`	22 `0xFFBF`	23 `0xFF7F`
24 `0xFEFF`	25 `0xFDFF`	26 `0xFBFF`	27 `0xF7FF`	28 `0xEFFF`	29 `0xDFFF`	30 `0xBFFF`	31 `0x7FFF`

immask5: value by field value

0 `0x000F`	1 `0x0003`	2 `0xFF00`	3 `0x000C`	4 `0x00F0`	5 `0x0030`	6 `0xF0F0`	7 `0x00C0`
8 `0x0F00`	9 `0x0300`	10 `0xCCCC`	11 `0x0C00`	12 `0xF000`	13 `0x3000`	14 `0xAAAA`	15 `0xC000`
16 `0xFFF0`	17 `0xFFFC`	18 `0x00FF`	19 `0xFFF3`	20 `0xFF0F`	21 `0xFFCF`	22 `0x0F0F`	23 `0xFF3F`
24 `0xF0FF`	25 `0xFCFF`	26 `0x3333`	27 `0xF3FF`	28 `0x0FFF`	29 `0xCFFF`	30 `0x5555`	31 `0x3FFF`

Same Test, Other Instruction

Some comparisons against a constant are bit tests: R[a] <u 256 holds exactly when none of bits 8–15 is set, and R[a] < 0 when bit 15 is. When the test written has no encoding of its own but the other instruction encodes the same test, the assembler uses that instead, in either direction - same length, same cycles, same branch. A disassembler prints what was encoded.

Written	Assembles as
`br lo, ra, #2`	`brclr ra, #0xFFFE`
`br ls, ra, #1`	`brclr ra, #0xFFFE`
`br lo, ra, #4`	`brclr ra, #0xFFFC`
`br ls, ra, #3`	`brclr ra, #0xFFFC`
`br lo, ra, #16`	`brclr ra, #0xFFF0`
`br ls, ra, #15`	`brclr ra, #0xFFF0`

Written	Assembles as
`br lo, ra, #256`	`brclr ra, #0xFF00`
`br ls, ra, #255`	`brclr ra, #0xFF00`
`br lo, ra, #4096`	`brclr ra, #0xF000`
`br ls, ra, #4095`	`brclr ra, #0xF000`
`br lo, ra, #16384`	`brclr ra, #0xC000`
`br ls, ra, #16383`	`brclr ra, #0xC000`
`br lo, ra, #32768`	`brclr ra, #0x8000`
`br ls, ra, #32767`	`brclr ra, #0x8000`
`br gt, ra, #-1`	`brclr ra, #0x8000`
`br ge, ra, #0`	`brclr ra, #0x8000`
`brclr ra, #0xFFFF`	`br eq, ra, #0`
`brclr ra, #0xFFE0`	`br lo, ra, #32`
`brclr ra, #0xFFF8`	`br lo, ra, #8`

Operation

```
if ((R[a] & mask) == 0) pc = pc + off;
```

Notes

Branch on a masked test of one register: ‘ra & mask‘ against zero.

```
brclr ra, #mask, target      taken when (ra & mask) == 0
brset ra, #mask, target      taken when (ra & mask) != 0
```

The mask is an immbit5, so the two of them together cover every single bit of every register in both senses - which is all they used to do - plus every ALL-BUT-ONE-BIT test, which is new and is what the fifth bit used to be spent on. ‘brclr r0, #0x7fff, L‘ branches when nothing but the sign bit is set.

THEY ARE FOUR OPCODES, a mask table each and a sense each: the mask feeds the ALU, the result is compared with zero, and the opcode carries the condition. Nothing new is needed in the compare path.

```
byte 0:  0xac, 0xad = brclr, immbit5 and immask5
         0xb4, 0xb5 = brset, the same
```

THE OPERAND IS A MASK, NOT A BIT NUMBER. ‘brset ra, #0x8000‘ tests the sign bit; ‘brset ra, #7‘ is not an instruction, because 7 is in neither immbit5 nor immask5, and the assembler rejects it rather than guessing. That catches most bit numbers written by mistake, but not all: 1, 2, 4 and 8 are single-bit masks and 3, 12 and 15 are immask5 entries, so ‘brset ra, #15‘ assembles - and tests the low four bits, not bit 15.

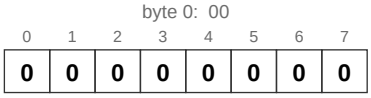
BRK — software interrupt: enter the handler

Opcode	Instruction	Bytes	Cycles	Form	Operands
`00`	`brk`	1	7	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **`00`** `brk`



Operation

```
shadow_lr = lr;
lr = pc;
shadow_sp = sp;
sp = shadow_isp;
ie = 0;
pc = vector;
```

Notes

THE TWO LOWEST OPCODES ARE brk AND halt - enter an exception, wait for one; rti, returning from one, is 0x09 - and they are at 0x00 and 0x01 for the reason halt used to be at zero on its own. Erased memory, an unwritten ROM and a wild jump into a zeroed page all land on 0x00, and the useful thing for the machine to do there is stop in a way somebody can find afterwards.

BRK IS STRICTLY BETTER AT ZERO THAN HALT WAS. A halt at the point of the mistake leaves the pc sitting on the instruction that was wrong, which is already far better than a nop sled - but it stops the machine dead and the only way to learn anything is to attach something to the bus. brk traps to the handler with the faulting address in lr, so a monitor can print it. The sled argument that put halt at zero is unchanged and now buys more.

NOP IS AWAY FROM THEM, at 0x82, which is the same argument run the other way: the low opcodes are what a mistake lands on, so they hold the instructions that stop or divert, and nop sits well away. A nop sled through erased memory produces a machine that looks alive while executing nothing, which is the harder failure to diagnose of the two.

THEY ARE ALL IN THE ONE-BYTE REGION, as everything one byte long is: ‘(op & 0x70) == 0’ is the one-byte test.

WHY THESE THREE WRITE THE PC FROM OUTSIDE THE CONTROL ROWS. Their rule is about PROGRAM control flow - the transfers a compiler emits, a disassembler follows and a decoder predicts. An exception is not that: it is not reached from the instruction stream and its destination is not in the instruction. So the rows keep their meaning and these keep theirs, and the map is read as ‘0xa8 .. 0xb7 is every transfer with an operand’ rather than as a claim about the pc with exceptions.

brk 1 byte

A SOFTWARE INTERRUPT, and it takes the same path as a hardware one: the same microcode routine, the same shadow registers, the same vector. What differs is only that it is reached by executing an opcode rather than by a pin, and so it is NOT maskable - ‘ie‘ gates the pin and nothing else. A syscall that could be ignored because the caller happened to be inside a critical section would be useless.

IT RESUMES AFTER ITSELF, where a hardware interrupt restarts the instruction it preempted. ‘pc‘ is advanced before the semantics run - see the semantics section at the top of this file - so ‘lr = pc‘ saves the address FOLLOWING the brk, and ‘rti‘ returns there. That is what a syscall wants. The hardware path saves the address of the instruction that had not started yet, so that one is restarted instead; both are ‘lr = pc‘ and the difference is entirely in which cycle the interrupt is taken in.

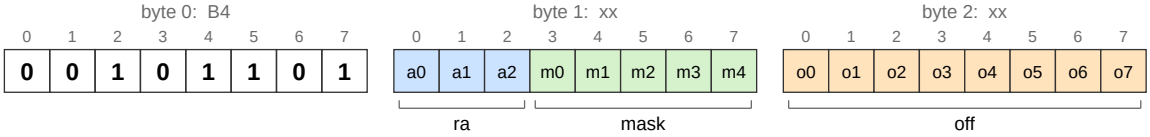
BRSET — branch if ra & mask is non-zero

Opcode	Instruction	Bytes	Cycles	Form	Operands
`B4 xx xx`	`brset ra, #immbit5, off8`	3	1, taken 3, wrong 5	reg_mask	immbit5: one of the 32 values below; off8: -128 to 127, sign-extended to 16 bits
`B5 xx xx`	`brset ra, #immask5, off8`	3	1, taken 3, wrong 5	reg_mask5	immask5: one of the 32 values below; off8: -128 to 127, sign-extended to 16 bits

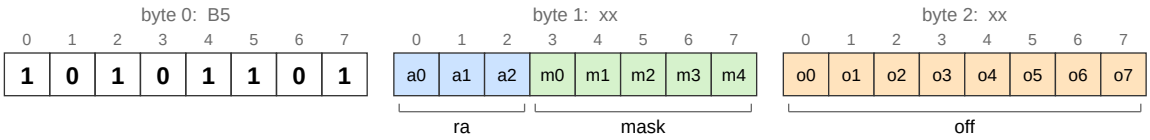
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_mask **B4 xx xx** `brset ra, #immbit5, off8`



reg_mask5 **B5 xx xx** `brset ra, #immask5, off8`



The same test against the mask table instead of a single bit - branch on whether a whole FIELD is empty. Same byte 1, same 5 bits, different table: +4 of the row reads them as immbit5 and +5 reads them as immask5, which is the relationship every ALU group has between its two mask forms. Clear and set are separated by BIT 3 rather than bit 0 - brclr is the upper row and brset the row eight below - so both spellings of each land in one column.

Operand Values

immbit5: value by field value

0 `0x0001`	1 `0x0002`	2 `0x0004`	3 `0x0008`	4 `0x0010`	5 `0x0020`	6 `0x0040`	7 `0x0080`
8 `0x0100`	9 `0x0200`	10 `0x0400`	11 `0x0800`	12 `0x1000`	13 `0x2000`	14 `0x4000`	15 `0x8000`
16 `0xFFFE`	17 `0xFFFD`	18 `0xFFFB`	19 `0xFFF7`	20 `0xFFEF`	21 `0xFFDF`	22 `0xFFBF`	23 `0xFF7F`
24 `0xFEFF`	25 `0xFDFF`	26 `0xFBFF`	27 `0xF7FF`	28 `0xEFFF`	29 `0xDFFF`	30 `0xBFFF`	31 `0x7FFF`

immask5: value by field value

0 `0x000F`	1 `0x0003`	2 `0xFF00`	3 `0x000C`	4 `0x00F0`	5 `0x0030`	6 `0xF0F0`	7 `0x00C0`
8 `0x0F00`	9 `0x0300`	10 `0xCCCC`	11 `0x0C00`	12 `0xF000`	13 `0x3000`	14 `0xAAAA`	15 `0xC000`
16 `0xFFF0`	17 `0xFFFC`	18 `0x00FF`	19 `0xFFF3`	20 `0xFF0F`	21 `0xFFCF`	22 `0x0F0F`	23 `0xFF3F`
24 `0xF0FF`	25 `0xFCFF`	26 `0x3333`	27 `0xF3FF`	28 `0x0FFF`	29 `0xCFFF`	30 `0x5555`	31 `0x3FFF`

Same Test, Other Instruction

Some comparisons against a constant are bit tests: R[a] <u 256 holds exactly when none of bits 8–15 is set, and R[a] < 0 when bit 15 is. When the test written has no encoding of its own but the other instruction encodes the same test, the assembler uses that instead, in either direction - same length, same cycles, same branch. A disassembler prints what was encoded.

Written	Assembles as
`br lt, ra, #0`	`brset ra, #0x8000`
`br le, ra, #-1`	`brset ra, #0x8000`
`br hi, ra, #32767`	`brset ra, #0x8000`
`br hs, ra, #32768`	`brset ra, #0x8000`
`br hi, ra, #1`	`brset ra, #0xFFFE`
`br hs, ra, #2`	`brset ra, #0xFFFE`

Written	Assembles as
`br hi, ra, #3`	`brset ra, #0xFFFC`
`br hs, ra, #4`	`brset ra, #0xFFFC`
`br hi, ra, #15`	`brset ra, #0xFFF0`
`br hs, ra, #16`	`brset ra, #0xFFF0`
`br hi, ra, #255`	`brset ra, #0xFF00`
`br hs, ra, #256`	`brset ra, #0xFF00`
`br hi, ra, #4095`	`brset ra, #0xF000`
`br hs, ra, #4096`	`brset ra, #0xF000`
`br hi, ra, #16383`	`brset ra, #0xC000`
`br hs, ra, #16384`	`brset ra, #0xC000`
`brset ra, #0xFFFF`	`br ne, ra, #0`
`brset ra, #0xFFE0`	`br hs, ra, #32`
`brset ra, #0xFFF8`	`br hs, ra, #8`

Operation

```
if ((R[a] & mask) != 0) pc = pc + off;
```

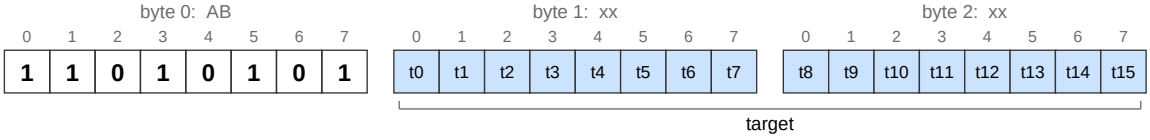
CALL — lr = pc of next instruction; pc = imm16; lr = pc of next instruction; pc = ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
`AB xx xx`	`call imm16`	3	3	abs16	imm16: any 16-bit value
`B3 00–07`	`call ra`	2	5	reg	
`03`	`call r5`	1	5	r5	All operands implicit

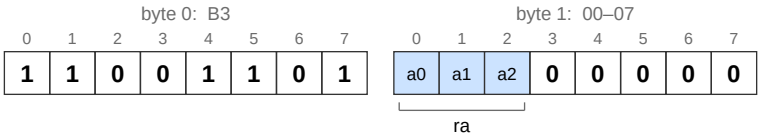
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

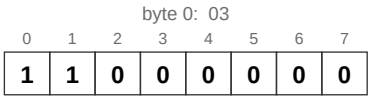
abs16 **AB xx xx** `call imm16`



reg **B3 00–07** `call ra`



r5 **03** `call r5`



Operation

call target

```
lr = pc;
pc = target;
```

call ra

```
lr = pc;
pc = R[a];
```

Notes

Return is just pc = lr, so it needs no operands and fits in one byte - and being one byte is what decides where it lives.

IT IS AT 0x08, OUTSIDE THE CONTROL ROW AT 1010_1xxx, deliberately, as ‘jmp r5‘ and ‘call r5‘ are at 0x02 and 0x03: every one-byte instruction is in 0000_xxxx or 1000_xxxx, those that take more than a cycle in the first and those that take one in the second, so that is where the three go.

It fits that region on its own terms rather than as an exile. The one-byte block is instructions with nothing left to say - ‘add r0, r0, #1‘, ‘mov r0, #0‘, ‘ld‘ with its register implied - and ‘ret‘ is ‘jmp lr‘ with its operand implied, which is the same kind of thing. It is also, after the ALU, the most executed instruction in any program, which is what a one-byte opcode is for.

Nothing decodes worse for it: decode is a flat 256-entry table and always was, so where the pc writers sit is a claim about how the map READS, not a mask any hardware applies - but for the ret’s guess, which picks out 0x08 whole.

‘call rX‘ is ‘call imm16‘ with the target coming from the register file instead of the instruction stream: same lr write, same jump, a byte less to fetch. Two bytes. Five cycles, as the target is known only in the ALU cycle.

IT IS NOT OPTIONAL FOR C. Before it existed, ‘ret‘ was the only transfer that read a register, so a function pointer could be tail called - ‘mov lr, rX‘ then ‘ret‘ - and nothing else. There was no way to arrange a return address, because the only instruction that could jump to a register was the one that consumed the register holding it. Function pointers, vtables and qsort all need this

opcode.

‘call lr‘ IS UNDEFINED. The semantics write lr before they read R[a], so taken literally it goes to the next instruction; the byte-serial core read the old lr and went there. Neither is worth a gate: the target of this one would be the lr it writes. gcc never emits it.

AND ‘call r5‘ IS ONE BYTE, as ‘jmp r5‘ is. r5 carries no argument and no return value, so a call through a pointer with its arguments in r0-r3 can have its target waiting there - a vtable call, or an interpreter’s dispatch written to use it. The general form stays for everything else.

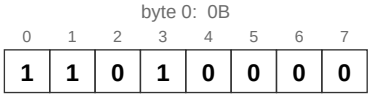
CLI — disable interrupts

Opcode	Instruction	Bytes	Cycles	Form	Operands
0B	cli	1	4	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **0B** cli



Operation

```
ie = 0;
```

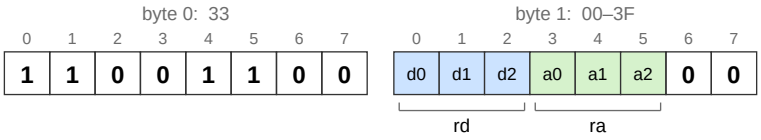
CLZ — rd = count of leading zeros in ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
33 00–3F	clz rd, ra	2	2	reg_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **33 00–3F** clz rd, ra



Operation

```
R[d] = clz(R[a]);
```

Takes 1 cycle beyond the one an ALU instruction costs.

Notes

Two cycles: its result is registered halfway. See ‘extra_cycles‘ in the cost model, and the slow pair above.

HALT — stop the machine until an interrupt

Opcode	Instruction	Bytes	Cycles	Form	Operands
`01`	`halt`	1	1	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **01** `halt`

byte 0: 01

0	1	2	3	4	5	6	7
1	**0**	**0**	**0**	**0**	**0**	**0**	**0**

Operation

```
halted = 1;
```

Notes

halt 1 byte

STOP UNTIL AN INTERRUPT, and with interrupts disabled that is stop forever - which is the old halt, unchanged, and is what a wild jump still finds. The two behaviours are one instruction because ‘ie‘ already distinguishes them.

IT RESUMES AFTER ITSELF, like the 65C02’s WAI. Dispatching the halt consumes its opcode, so the pc is already past it while the machine waits, and the interrupt saves the address FOLLOWING the halt in lr - as brk does, and for the same reason: ‘pc‘ is advanced before the semantics run. An ‘rti‘ goes on from there.

SO THE POLICY IS THE IDLE LOOP’S. One that wants to keep sleeping through an interrupt says so, in two bytes: ‘idle: halt‘ and then ‘br idle‘, or a test of whatever the handler left behind before branching back. Making the interrupt save the halt’s own address instead, so that rti waited again, would need the pc less one on the path that saves it: a decrementer, a mux and a flop to say the entry came from a halt.

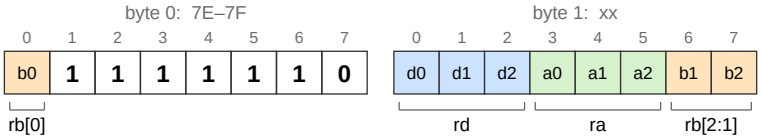
ISEQ — rd = 1 if ra equals rb, else 0; rd = 1 if ra equals a constant, else 0

Opcode	Instruction	Bytes	Cycles	Form	Operands
7E–7F xx	iseq rd, ra, rb	2	1	three_reg	
78 xx	iseq rd, rd, #imm5	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
7A–7B xx	iseq rd, ra, #imm3	2	1	reg_reg_imm3	imm3: one of the 8 values below
79 xx xx	iseq rd, ra, #imm10	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

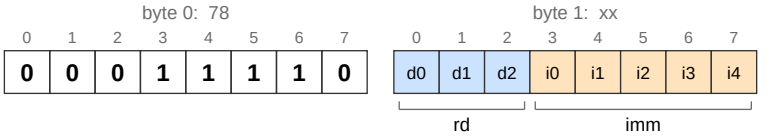
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

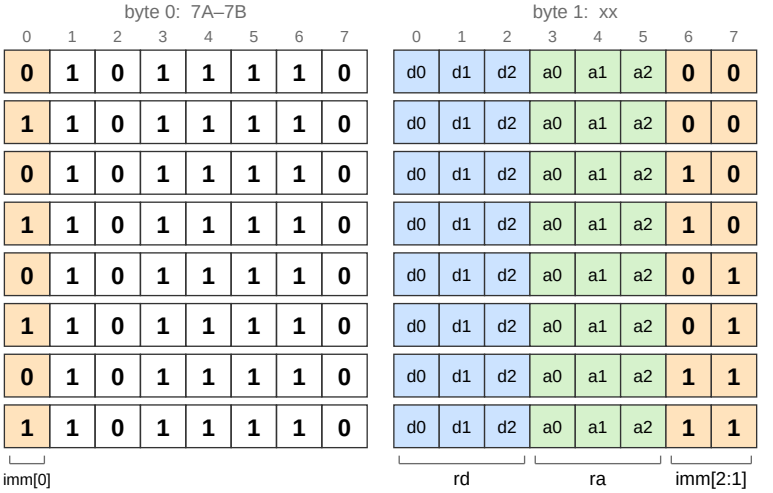
three_reg **7E–7F xx** iseq rd, ra, rb



reg_imm5 **78 xx** iseq rd, rd, #imm5

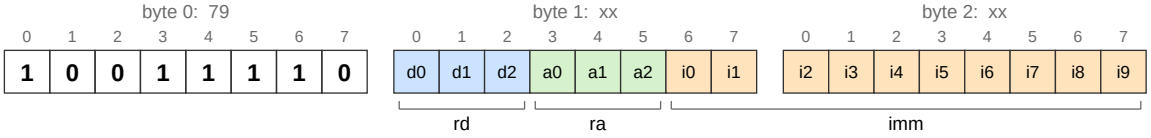


reg_reg_imm3 **7A–7B xx** iseq rd, ra, #imm3



7A 00–3F	iseq rd, ra, #-1	
7B 00–3F	iseq rd, ra, #0	
7A 40–7F	iseq rd, ra, #1	
7B 40–7F	iseq rd, ra, #2	
7A 80–BF	iseq rd, ra, #3	
7B 80–BF	iseq rd, ra, #4	
7A C0–FF	iseq rd, ra, #6	
7B C0–FF	iseq rd, ra, #8	

reg_reg_imm10 **79 xx xx** iseq rd, ra, #imm10



Operand Values

imm3: value by field value

0 -1 1 0 2 1 3 2 4 3 5 4 6 6 7 8

Operation

iseq rd, ra, rb

```
R[d] = R[a] == R[b];
```

iseq rd, ra, #imm

```
R[d] = R[a] == (imm & 0xffff);
```

imm5 and imm10 are sign-extended to 16 bits before use.

Notes

The immediate is compared as a 16-bit PATTERN. Written ‘R[a] == imm’ this compared a register, which is 0..65535, against the signed value decoded from imm10 - so ‘iseq r0, r0, #-205’ was false for r0 = -205, where the comparator sees 0xff33 on both ports. The compiler found it.

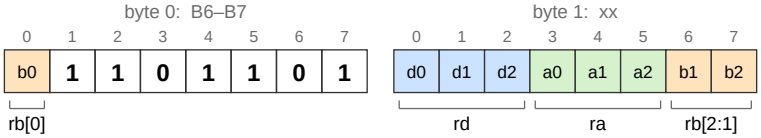
ISHI — rd = 1 if ra is higher than rb, unsigned, else 0

Opcode	Instruction	Bytes	Cycles	Form	Operands
B6–B7 xx	ishi rd, ra, rb	2	1	three_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **B6–B7 xx** ishi rd, ra, rb



Operation

```
R[d] = R[a] > R[b];
```

ISLO — rd = 1 if ra is lower than rb, unsigned, else 0

An assembler alias: it has no encoding of its own.

Assembler Aliases

Written	Assembles as	Description
`islo rd, ra, rb`	`ishi rd, rb, ra`	rd = 1 if ra is lower than rb, unsigned, else 0

Encodings: see ISHI.

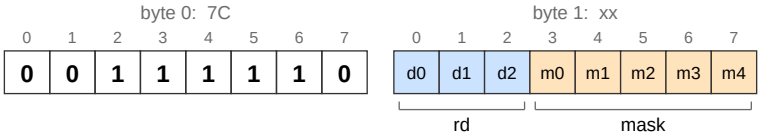
ISSET — rd = 1 if ra and the mask share a bit, else 0

Opcode	Instruction	Bytes	Cycles	Form	Operands
`7C xx`	`isset rd, rd, #immbit5`	2	1	reg_immbit5	immbit5: one of the 32 values below; ra is rd
`7D xx`	`isset rd, rd, #immask5`	2	1	reg_immask5	immask5: one of the 32 values below; ra is rd

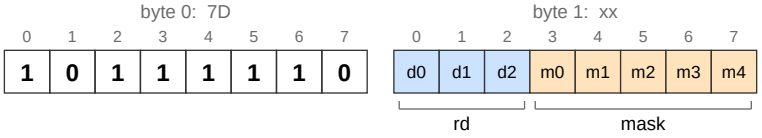
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_immbit5 **`7C xx`** `isset rd, rd, #immbit5`



reg_immask5 **`7D xx`** `isset rd, rd, #immask5`



Operand Values

immbit5: value by field value

0	`0x0001`	1	`0x0002`	2	`0x0004`	3	`0x0008`	4	`0x0010`	5	`0x0020`	6	`0x0040`	7	`0x0080`
8	`0x0100`	9	`0x0200`	10	`0x0400`	11	`0x0800`	12	`0x1000`	13	`0x2000`	14	`0x4000`	15	`0x8000`
16	`0xFFFE`	17	`0xFFFD`	18	`0xFFFB`	19	`0xFFF7`	20	`0xFFEF`	21	`0xFFDF`	22	`0xFFBF`	23	`0xFF7F`
24	`0xFEFF`	25	`0xFDFF`	26	`0xFBFF`	27	`0xF7FF`	28	`0xEFFF`	29	`0xDFFF`	30	`0xBFFF`	31	`0x7FFF`

immask5: value by field value

0	`0x000F`	1	`0x0003`	2	`0xFF00`	3	`0x000C`	4	`0x00F0`	5	`0x0030`	6	`0xF0F0`	7	`0x00C0`
8	`0x0F00`	9	`0x0300`	10	`0xCCCC`	11	`0x0C00`	12	`0xF000`	13	`0x3000`	14	`0xAAAA`	15	`0xC000`
16	`0xFFF0`	17	`0xFFFC`	18	`0x00FF`	19	`0xFFF3`	20	`0xFF0F`	21	`0xFFCF`	22	`0x0F0F`	23	`0xFF3F`
24	`0xF0FF`	25	`0xFCFF`	26	`0x3333`	27	`0xF3FF`	28	`0x0FFF`	29	`0xCFFF`	30	`0x5555`	31	`0x3FFF`

Operation

```
R[d] = (R[a] & mask) != 0;
```

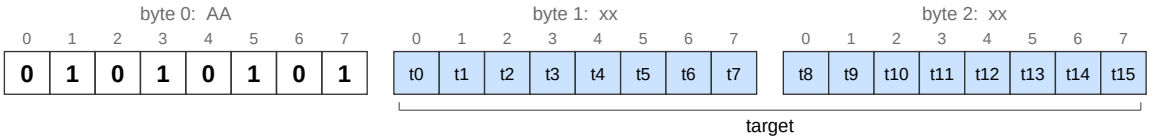
JMP — pc = imm16; pc = ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
AA xx xx	jmp imm16	3	2	abs16	imm16: any 16-bit value
B2 00–07	jmp ra	2	5	reg	
02	jmp r5	1	5	r5	All operands implicit

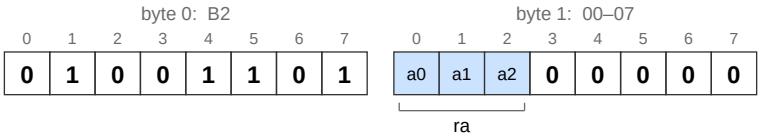
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

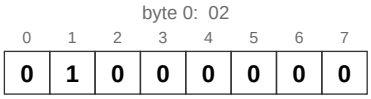
abs16 **AA xx xx** jmp imm16



reg **B2 00–07** jmp ra



r5 **02** jmp r5



Operation

jmp target

```
pc = target;
```

jmp ra

```
pc = R[a];
```

Notes

Five opcodes. The wide immediate is the ‘mov rd, #imm16‘ shape with the register field spent on the opcode instead, so these reuse the same fetch path: opcode byte, then a 16-bit value low byte first.

Relative and absolute both take a full 16 bits and both reach the whole address space, so the choice between them is not about range - it is whether the code is position independent, and nothing here is.

THE WIDE RELATIVE FORMS ARE GONE, and what they cost is why. ‘jmpr target’ and ‘callr target’ were three bytes reaching the whole address space relatively; ‘jmp’ and ‘call’ are three bytes reaching it absolutely. On the byte-serial core a relative target cost a CYCLE MORE, because the adder and the verdict landed in flops an edge later than the instruction’s own bytes did: three bytes and four cycles against three and three. On the pipeline the absolute form reads its target as it dispatches, where a relative one would need an adder in dispatch to keep up, so what they would cost now is that adder and two opcodes, for position independence that a machine with a linker and no address-space randomisation never asked for.

So jmp and call sit side by side at 0xaa and 0xab, with the same byte 1, each above its form through a register: jmp ra at 0xb2, call ra at 0xb3. Code that must run at an address it was not linked for can jump only through a register.

THE SHORT RELATIVE JUMP IS GONE TOO. ‘jmpr’ was two bytes at 0xb3, where call ra is now - the compare-and-branch datapath with its condition pinned to ‘eq, r0, r0’, and the only two-byte relative transfer: so decode chose a displacement from byte 1 or byte 2 for it alone, gas relaxed it and nothing else, and a predictor had one more shape to recognise. What it saved was a byte on a local jump; gcc now emits ‘jmp’, a byte longer and as fast.

A JUMP THROUGH A REGISTER, WHICH THE MACHINE OTHERWISE HAS NO WAY TO SPELL. Without it an indirect jump has to go ‘mov lr, ra‘ and ‘ret‘, which costs three bytes and, far worse, destroys lr - so a function with a switch table had to save lr as

if it made calls, and a tail call through a function pointer was impossible outright, since lr is holding the return address the tail call exists to pass on.

IT TAKES ANY REGISTER, IN TWO BYTES, AND r5 IN ONE. It was once one byte with r5 pinned and nothing else, and the byte it saved was charged back in register allocation: every indirect jump had to get its address into the one register the assembler is also free to clobber. Both forms cost the same, five cycles, because what this instruction pays for is the register, not fetching a byte. With both, nothing has to move for the general case, and code that can choose r5 - a tail call through a pointer, which gcc stages in r5 anyway, or an interpreter's dispatch - saves the byte.

ret IS ONE BYTE TOO: 'pc = lr' needs no operand at all. The three of them sit in the one-byte region, at 0x08, 0x02 and 0x03, with every other instruction of their length. ret used to be at 1010_1011, in the control row, so that the scarce one-byte region kept its slots and every transfer of control was 1010_xxxx; with the region twice the size and its length test simpler for holding everything one byte long, that was the wrong trade.

jmp r5 and call r5 are 0x02 and 0x03 for their low bits, which are jmp ra's and call ra's at 0xb2 and 0xb3: a jmp ra predictor, since removed, told all four from everything else with a few bits. 0x00 and 0x01 are brk and halt, so a wild jump into a zeroed page traps to a handler that can name the faulting address rather than unwinding to wherever lr happened to point.

LD — load 16-bit; load 16-bit through a register index

Opcode	Instruction	Bytes	Cycles	Form	Operands
`04`	`ld r0, [r0, #0]`	1	3, odd 4	implicit	All operands implicit
`06`	`ld r0, [r6, #0]`	1	3, odd 4	r0_sp	All operands implicit
`20 xx`	`ld ra, [ra, #imm5]`	2	3, odd 4	reg_off5	imm5: -16 to 15, sign-extended to 16 bits; rd is ra
`22–23 xx`	`ld rd, [ra, #imm3]`	2	3, odd 4	reg_reg	imm3: one of the 8 values below
`21 xx xx`	`ld rd, [ra, #imm10]`	3	3, odd 4	reg_reg10	imm10: -512 to 511, sign-extended to 16 bits
`26–27 xx`	`ld rd, [ra, rb]`	2	3, odd 4	reg_reg_reg	

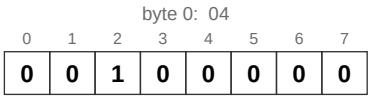
Assembler Aliases

Written	Assembles as	Description
`ld rd, [ra]`	`ld rd, [ra, #0]`	rd = memory16[ra]

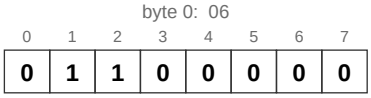
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

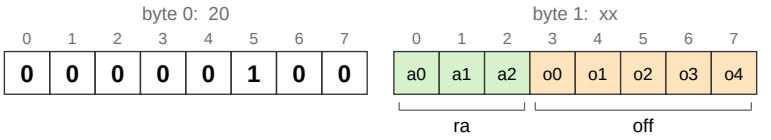
implicit **`04`** `ld r0, [r0, #0]`



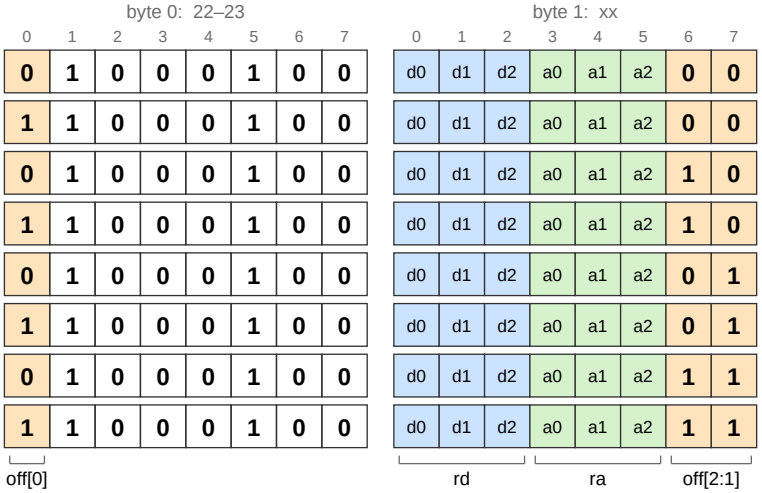
r0_sp **`06`** `ld r0, [r6, #0]`



reg_off5 **`20 xx`** `ld ra, [ra, #imm5]`

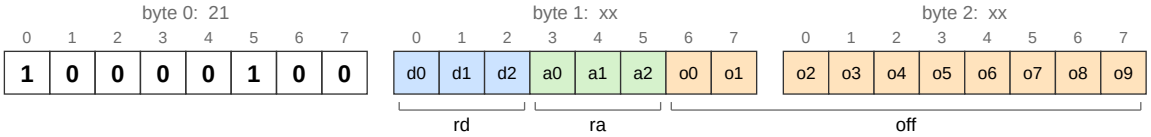


reg_reg **22–23 xx** `ld rd, [ra, #imm3]`

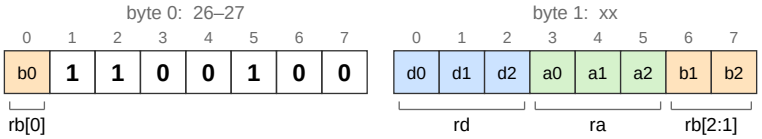


22 00–3F	`ld rd, [ra, #-1]`	
23 00–3F	`ld rd, [ra, #0]`	
22 40–7F	`ld rd, [ra, #1]`	
23 40–7F	`ld rd, [ra, #2]`	
22 80–BF	`ld rd, [ra, #3]`	
23 80–BF	`ld rd, [ra, #4]`	
22 C0–FF	`ld rd, [ra, #6]`	
23 C0–FF	`ld rd, [ra, #8]`	

reg_reg10 **21 xx xx** `ld rd, [ra, #imm10]`



reg_reg_reg **26–27 xx** `ld rd, [ra, rb]`



Operand Values

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

ld rd, [ra, #off]

```
R[d] = M16[R[a] + off];
```

ld rd, [ra, rb]

```
R[d] = M16[R[a] + R[b]];
```

Notes

Load a 16-bit value. Structurally identical to ld8 - same four addressing forms, same operand shape, same field layouts - differing only in opcode bit 3, which is the width bit for the whole load block.

DECIDED: displacements count BYTES and are NOT scaled by the access width, for ld and st alike. Scaling would have doubled the reach of the off5 and imm10 forms for almost no hardware cost, and it is still rejected, because POINTERS ARE TAGGED. A heap reference carries a tag in its low bit, so the displacement is what removes it:

```
ld rd, [rp, #-1]     first word of an object behind a tag-1 pointer
ld rd, [rp, #1]      second word
ld rd, [rp, #3]      third
```

Those displacements are ODD. A scaled offset cannot express them at all, and would put a shift or an add in front of every single field access. Byte displacements are not a conservative default here; they are load bearing.

ALSO DECIDED: unaligned accesses are allowed. An odd effective address reads or writes two consecutive bytes, in the ‘endian‘ order declared at the top, with no fault and no penalty visible to software. Given that displacements do the tag arithmetic, a correct field access lands aligned anyway - this just means the hardware never has to say no.

LD8 — load 8-bit, zero extended; load 8-bit through a register index, zero extended

Opcode	Instruction	Bytes	Cycles	Form	Operands
`05`	`ld8 r0, [r0, #0]`	1	3	implicit	All operands implicit
`28 xx`	`ld8 ra, [ra, #imm5]`	2	3	reg_off5	imm5: -16 to 15, sign-extended to 16 bits; rd is ra
`2A–2B xx`	`ld8 rd, [ra, #imm3]`	2	3	reg_reg	imm3: one of the 8 values below
`29 xx xx`	`ld8 rd, [ra, #imm10]`	3	3	reg_reg10	imm10: -512 to 511, sign-extended to 16 bits
`2E–2F xx`	`ld8 rd, [ra, rb]`	2	3	reg_reg_reg	

Assembler Aliases

Written	Assembles as	Description
`ld8 rd, [ra]`	`ld8 rd, [ra, #0]`	rd = memory8[ra], zero extended

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

implicit **05** `ld8 r0, [r0, #0]`

byte 0: 05

0	1	2	3	4	5	6	7
1	0	1	0	0	0	0	0

reg_off5 **28 xx** `ld8 ra, [ra, #imm5]`

byte 0: 28

0	1	2	3	4	5	6	7
0	0	0	1	0	1	0	0

byte 1: xx

0	1	2	3	4	5	6	7
a0	a1	a2	o0	o1	o2	o3	o4
ra			off				

reg_reg **2A–2B xx** `ld8 rd, [ra, #imm3]`

byte 0: 2A–2B

byte 1: xx

byte 0								byte 1									
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7		
0	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	0	0	**2A 00–3F**	`ld8 rd, [ra, #-1]`
1	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	0	0	**2B 00–3F**	`ld8 rd, [ra, #0]`
0	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	1	0	**2A 40–7F**	`ld8 rd, [ra, #1]`
1	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	1	0	**2B 40–7F**	`ld8 rd, [ra, #2]`
0	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	0	1	**2A 80–BF**	`ld8 rd, [ra, #3]`
1	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	0	1	**2B 80–BF**	`ld8 rd, [ra, #4]`
0	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	1	1	**2A C0–FF**	`ld8 rd, [ra, #6]`
1	1	0	1	0	1	0	0	d0	d1	d2	a0	a1	a2	1	1	**2B C0–FF**	`ld8 rd, [ra, #8]`
off[0]								rd			ra			off[2:1]			

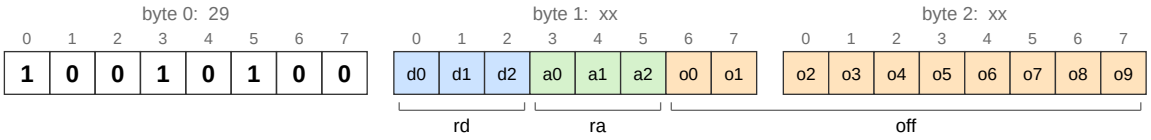
The 3-bit table immediate is split across the instruction: its LOW bit is the low bit of the opcode byte, so this form burns two adjacent opcodes (0x2a and 0x2b). Its other two bits sit at the bottom of byte 1.

Paying an opcode bit to widen the register fields is the whole trick here: it is what makes room for a separate destination and address in two bytes.

'first_for' MAKES THIS FORM WIN A TIE at the values it names: where an earlier form of the same size also fits - reg_off5, when d is a -

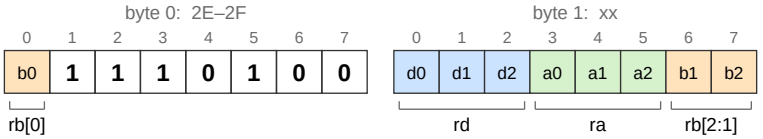
the assembler takes this one. So every load at offset 0 that is not one byte is this form, and a core can find one from byte 0 and the top two bits of byte 1 alone, whatever its registers, without decoding imm5.

reg_reg10 **29 xx xx** `ld8 rd, [ra, #imm10]`



Three bytes, and in exchange everything is independent and nothing is scattered: a whole opcode byte, then dest, address and a signed 10-bit displacement laid out in significance order.

reg_reg_reg **2E–2F xx** `ld8 rd, [ra, rb]`



Operand Values

imm3: value by field value

0	1	2	3	4	5	6	7
-1	0	1	2	3	4	6	8

Operation

ld8 rd, [ra, #off]

```
R[d] = M8[R[a] + off];
```

ld8 rd, [ra, rb]

```
R[d] = M8[R[a] + R[b]];
```

Notes

Load an 8-bit value, zero-extended into a 16-bit register.

Four encodings, in increasing size. The assembler tries them in the order listed and takes the first whose constraints the written operands satisfy:

```
implicit    1 byte   everything pinned: dest r0, address r0, offset 0
reg_off5    2 bytes  wide offset, but destination is TIED to the address
reg_reg     2 bytes  separate destination and address, but the offset must
                     come from the 8-entry imm3 table
reg_reg10   3 bytes  separate registers AND a -512..+511 offset
```

The two-byte forms trade against each other rather than one dominating: one buys offset range at the cost of a register, the other buys a register at the cost of offset range. The three-byte form buys both.

imm10 is deliberately where the ladder stops. If a displacement will not fit in -512..+511, the compiler computes the address into a spare register first; with eight registers that is cheap, and it saves the encoding from growing a four-byte tier that would earn its keep only rarely. So the assembler CAN legitimately reject an ‘ld8‘ - a displacement out of imm10 range is a source error, not a missing form.

fff = 11i, the last of the memory block’s addressing forms and the only one whose displacement is a REGISTER. Two opcodes each, for the reason the imm3 forms take two: a third register needs three bits and byte 1 has two to spare, so the low bit rides the opcode. The index is {byte1[7:6], opcode[0]} - the same three bits, in the same places, that the three-register ALU forms use, which is why nothing in the datapath is new: the address register arrives on port A, the index on port B, and the adder that was already computing ‘R[a] + off‘ computes ‘R[a] + R[b]‘ instead.

THE INDEX IS UNSCALED, at both widths and for loads and stores alike. That is the same decision the displacements took and it is the same argument: pointers are tagged, so the arithmetic that lands on a field is odd as often as not, and a scaled index cannot express an odd address at all. Scaling would also put a shift in front of the adder for every indexed access, including the untagged ones that would never use it.

THESE ARE SEPARATE ENTRIES RATHER THAN FORMS, because a form shares its instruction’s operand list: ‘off‘ is an int16 and ‘b‘ is a register, and no one entry can say both. The mnemonic is the same and the assembler chooses on the syntax, which is unambiguous - ‘#3‘ is a displacement and ‘r3‘ is an index. add, jmp and call are split the same way for the same reason.

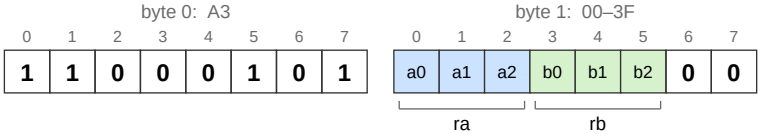
LDM — load ra then rb through r1; load ra, then rb, then rc through r1

Opcode	Instruction	Bytes	Cycles	Form	Operands
A3 00–3F	ldm ra, rb	2	4, odd 6	double	
A4–A5 xx	ldm ra, rb, rc	2	5, odd 8	triple	

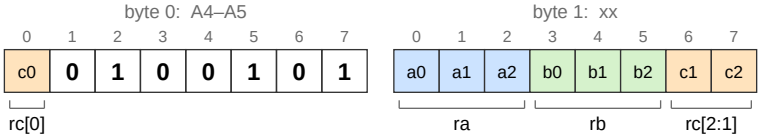
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

double **A3 00–3F** ldm ra, rb



triple **A4–A5 xx** ldm ra, rb, rc



Operation

ldm ra, rb

```
base = r1;
R[a] = M16[base];
R[b] = M16[base + 2];
r1 = base + 4;
```

ldm ra, rb, rc

```
base = r1;
R[a] = M16[base];
R[b] = M16[base + 2];
R[c] = M16[base + 4];
r1 = base + 6;
```

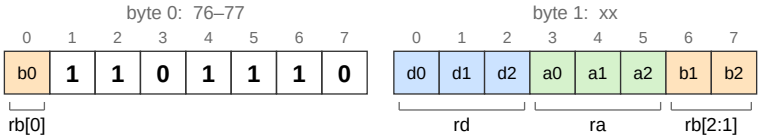
LSR — rd = ra >> (rb & 15), zero filling; rd = ra >> (imm & 15), zero filling

Opcode	Instruction	Bytes	Cycles	Form	Operands
`76–77 xx`	`lsr rd, ra, rb`	2	1	three_reg	
`70 00–7F`	`lsr rd, rd, #imm5`	2	1	reg_imm5	imm5: 0 to 15; bit 4 ignored, written 0; ra is rd
`72–73 xx`	`lsr rd, ra, #shift3`	2	1	reg_reg_imm3	shift3: one of the 8 values below
`71 xx 00-03`	`lsr rd, ra, #imm10`	3	1	reg_reg_imm10	imm10: 0 to 15; bits 9:4 ignored, written 0

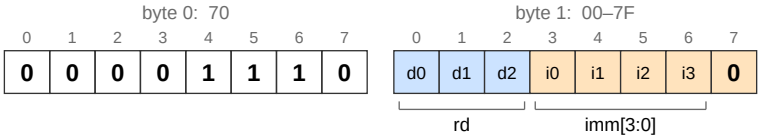
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

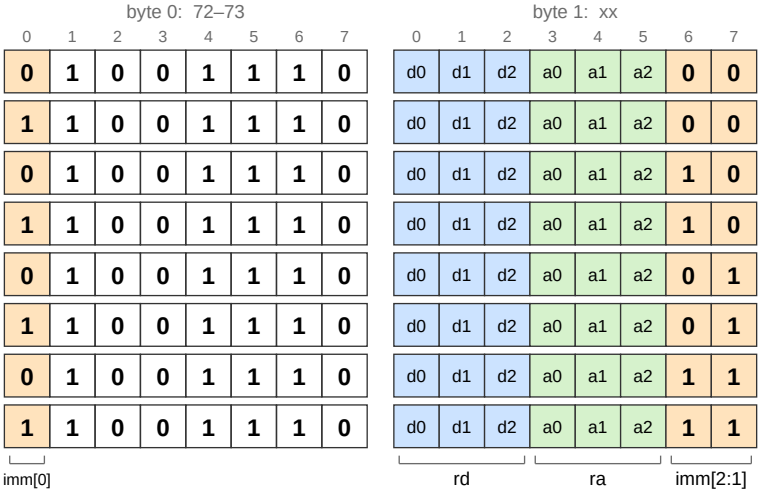
three_reg **76–77 xx** `lsr rd, ra, rb`



reg_imm5 **70 00–7F** `lsr rd, rd, #imm5`

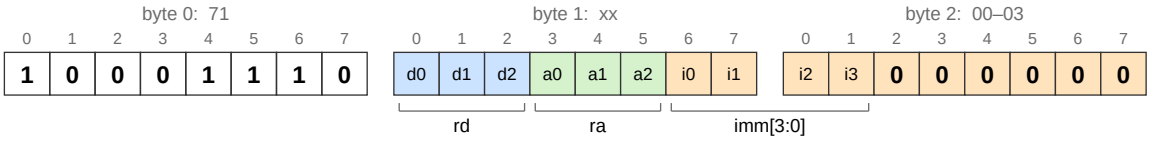


reg_reg_imm3 **72–73 xx** `lsr rd, ra, #shift3`



72 00–3F	`lsr rd, ra, #15`	
73 00–3F	`lsr rd, ra, #0`	
72 40–7F	`lsr rd, ra, #1`	
73 40–7F	`lsr rd, ra, #2`	
72 80–BF	`lsr rd, ra, #3`	
73 80–BF	`lsr rd, ra, #4`	
72 C0–FF	`lsr rd, ra, #6`	
73 C0–FF	`lsr rd, ra, #8`	

reg_reg_imm10 **71 xx 00-03** `lsr rd, ra, #imm10`



Operand Values

shift3: value by field value

0	15	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

lsr rd, ra, rb

```
R[d] = lsr(R[a], R[b] & 15);
```

lsr rd, ra, #imm

```
R[d] = lsr(R[a], imm & 15);
```

MOV — rd = immediate; rd = ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
88	mov r0, #0	1	1	r0_zero	All operands implicit
8B	mov r3, #0	1	1	r3_zero	All operands implicit
8A	mov r2, #0	1	1	r2_zero	All operands implicit
89	mov r1, #0	1	1	r1_zero	All operands implicit
87	mov r0, #1	1	1	r0_one	All operands implicit
30 xx	mov rd, #imm5	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits
34 xx	mov rd, #immbit5	2	1	reg_immbit5	immbit5: one of the 32 values below
35 xx	mov rd, #immask5	2	1	reg_immask5	immask5: one of the 32 values below
90–97 xx xx	mov rd, #imm16	3	1	reg_imm16	imm16: any 16-bit value
84	mov r0, r1	1	1	or.mov_r0_r1	Same encoding as or r0, r1, #0
85	mov r1, r0	1	1	or.mov_r1_r0	Same encoding as or r1, r0, #0
86	mov r2, r0	1	1	or.mov_r2_r0	Same encoding as or r2, r0, #0
53 00–3F	mov rd, ra	2	1	or.reg_reg_imm3	Same encoding as or rd, ra, #0

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

r0_zero **88** `mov r0, #0`

byte 0: 88

0	1	2	3	4	5	6	7
0	0	0	1	0	0	0	1

ZEROING r0 IS ONE BYTE. The 65C02 added STZ for exactly this reason - clearing a location was the commonest thing a program did that the 6502 had no short way to say - and setting a register to zero is the same observation one level in. Loop counters, accumulators, null pointers and cleared flags are all this instruction, and r0 is where a return value and a first argument live, so it is the register they most often want.

It costs one of the free one-byte slots and pins both operands, which is what an abbreviation is: a form of ‘mov‘ with nothing left to say. Three are left after this and ‘ret‘.

r3_zero **8B** `mov r3, #0`

byte 0: 8B

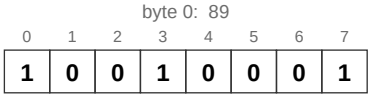
0	1	2	3	4	5	6	7
1	1	0	1	0	0	0	1

r2_zero **8A** `mov r2, #0`

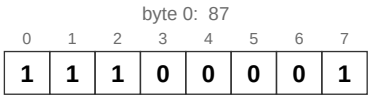
byte 0: 8A

0	1	2	3	4	5	6	7
0	1	0	1	0	0	0	1

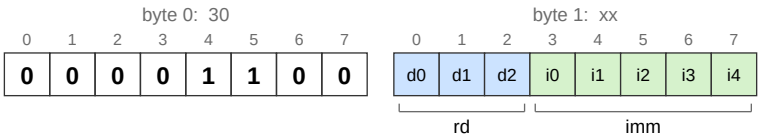
r1_zero **89** `mov r1, #0`



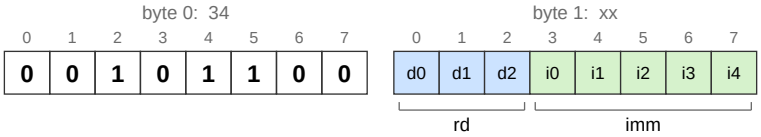
r0_one **87** `mov r0, #1`



reg_imm5 **30 xx** `mov rd, #imm5`



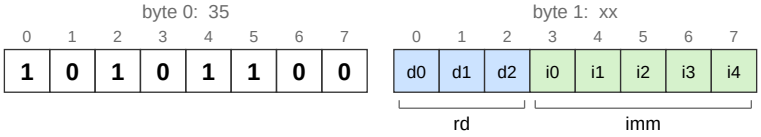
reg_immbit5 **34 xx** `mov rd, #immbit5`



The same two-byte shape, the same five bits, read through immbit5 instead of as a signed integer. That is what puts every power of two - and every value one bit short of all-ones - into two bytes rather than three.

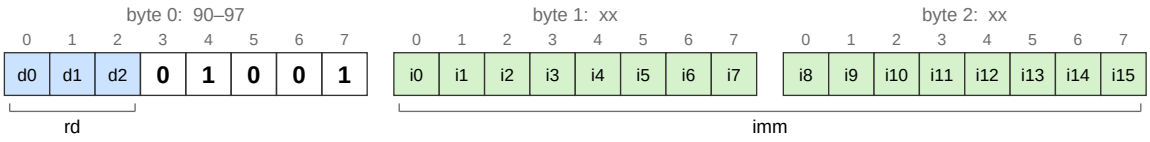
imm5 already reaches 1, 2, 4 and 8, so the assembler drops those entries from this form; what it is really buying is 16 upwards, 0x8000, and the complements from -17 down. ‘mov rd, #16‘ was three bytes before this form existed, which is a bad cliff to have one past the end of a signed 5-bit field.

reg_immask5 **35 xx** `mov rd, #immask5`



And the mask table. Loading 0x00ff or 0xf000 was three bytes before this; a routine that masks with a constant usually has to build it first.

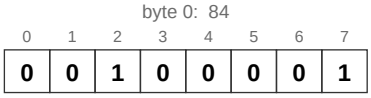
reg_imm16 **90–97 xx xx** `mov rd, #imm16`



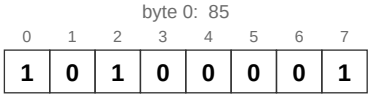
Destination in the opcode byte, then a whole 16-bit immediate. No narrowing happens here - the encoded type is the logical type - which is the one case where a form can represent everything the programmer writes.

The immediate is stored LOW BYTE FIRST, matching ‘endian‘, so the fetch unit assembles a 16-bit instruction immediate exactly the way it assembles a 16-bit data word. The bit map is written most-significant-first, so saying that takes an explicit split: ‘i‘ carries bits 7..0 and ‘j‘ carries bits 15..8.

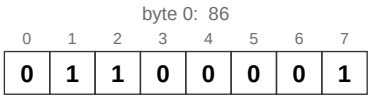
or.mov_r0_r1 **84** `mov r0, r1`



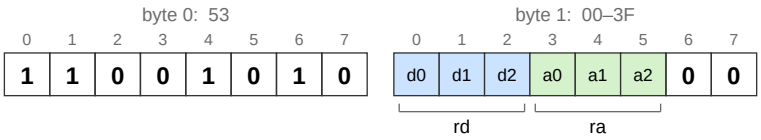
or.mov_r1_r0 **85** `mov r1, r0`



or.mov_r2_r0 **86** `mov r2, r0`



or.reg_reg_imm3 **53 00–3F** `mov rd, ra`



Operand Values

immbit5: value by field value

0	0x0001	1	0x0002	2	0x0004	3	0x0008	4	0x0010	5	0x0020	6	0x0040	7	0x0080
8	0x0100	9	0x0200	10	0x0400	11	0x0800	12	0x1000	13	0x2000	14	0x4000	15	0x8000
16	0xFFFE	17	0xFFFD	18	0xFFFB	19	0xFFF7	20	0xFFEF	21	0xFFDF	22	0xFFBF	23	0xFF7F
24	0xFEFF	25	0xFDFF	26	0xFBFF	27	0xF7FF	28	0xEFFF	29	0xDFFF	30	0xBFFF	31	0x7FFF

immask5: value by field value

0	0x000F	1	0x0003	2	0xFF00	3	0x000C	4	0x00F0	5	0x0030	6	0xF0F0	7	0x00C0
8	0x0F00	9	0x0300	10	0xCCCC	11	0x0C00	12	0xF000	13	0x3000	14	0xAAAA	15	0xC000
16	0xFFF0	17	0xFFFC	18	0x00FF	19	0xFFF3	20	0xFF0F	21	0xFFCF	22	0x0F0F	23	0xFF3F
24	0xF0FF	25	0xFCFF	26	0x3333	27	0xF3FF	28	0x0FFF	29	0xCFFF	30	0x5555	31	0x3FFF

Operation

mov rd, #imm

```
R[d] = imm;
```

mov rd, ra

```
R[d] = R[a] | 0;
```

Notes

mov rd, ra has no opcode of its own: it is an assembler alias for or rd, ra, #0, and shares that instruction's encodings, so a disassembler prints it as or.

There is no register-to-register move opcode. 'or rd, rs, #0' is a copy, and 0 is in the imm3 table, so the two-register form is two bytes for free. When the registers are r0 and r1 the pinned one-byte forms on 'or' take over, so 'mov r0, r1' costs a single byte - all without a mov opcode existing.

This is two mechanisms composing: the alias rewrites to 'or', and then shortest-first form selection finds the implicit encoding. Neither knows about the other.

Two forms. The short one is a register and a signed 5-bit immediate, laid out exactly like an ALU +0 form. The wide one carries a full 16-bit value.

The wide form costs EIGHT opcodes, and that is the cheap option rather than the expensive one. Three bytes is 24 bits; 16 of immediate plus 3 of register leave only 5 for the opcode, so the destination has to live in the opcode byte. Giving it a full 8-bit opcode instead would push the instruction to four bytes to say the same thing.

The imm16 field is the general wide-immediate form, and jmp and call will want it too - so those eight opcodes buy an addressing shape, not just a mov.

There is no register-to-register mov opcode; that is a preferred alias onto 'or', defined at the end of this file.

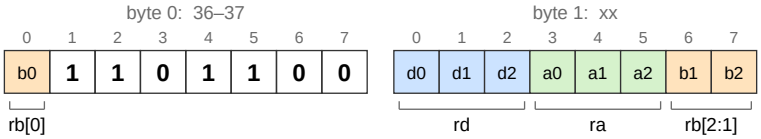
MOVHI — rd = (rb << 8) | (ra & 0xff); rd = (immediate << 8) | (ra & 0xff)

Opcode	Instruction	Bytes	Cycles	Form	Operands
36–37 xx	movhi rd, ra, rb	2	1	three_reg	
31 xx 00–3F	movhi rd, ra, #imm10	3	1	reg_reg_imm10	imm10: 0 to 255; bits 9:8 ignored, written 0

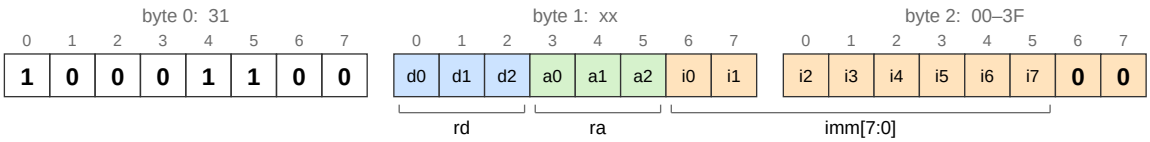
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **36–37 xx** movhi rd, ra, rb



reg_reg_imm10 **31 xx 00–3F** movhi rd, ra, #imm10



Operation

movhi rd, ra, rb

```
R[d] = ((R[b] & 0xff) << 8) | (R[a] & 0xff);
```

movhi rd, ra, #imm

```
R[d] = ((imm & 0xff) << 8) | (R[a] & 0xff);
```

Notes

ra with its high byte replaced: rd = (x << 8) | (ra & 0xff), where x is the low byte of rb or of the immediate. One ALU operation - the result's high byte is rhs[7:0] and its low byte lhs[7:0] - so the two forms share it, and they sit in the mov row's imm10 and three-register columns, 0x31 and 0x36/0x37, with exactly the byte-1 layouts those columns have everywhere else. No addressing mode is new: the immediate is imm10, and the operation reads its low eight bits.

WHAT IT IS FOR. A register's low byte and something else's in one register:

- the 6502's page-indexed addressing: with a 256-byte table on a page boundary, 'movhi r, idx, #page' then 'ld8 r, [r]' is the lookup, and it cleans the index's high byte on the way.
- the three-register form packs two variables' low bytes into one register, for anything that works on byte pairs.

THE IMMEDIATE GOES HIGH AND THE REGISTER LOW, and not the other way round. A constant under a shifted register - '(ra << 8) | #k' - is rare, and '#0' would only repeat 'shl rd, ra, #8'. This order is page indexing, and zero extension and setting the high byte in one, where the alternative is 'and', 'mov r5' and 'add'.

IT COSTS THREE CELLS: one more arm on the ALU's result mux, wiring into it. Measured on the processor behind one SPRAM, medians of sixteen seeds, 1345 LUT4 at 29.80 MHz before and 1348 at 29.40 after - a move well inside the spread of 27.7 .. 30.6, so no clock the seeds can see.

It is not a 'movhi' in the sense of a load-high-immediate: with the low byte also constant that is 'mov rd, #imm16', which exists. And '#0' is zero extension, which 'and rd, rd, #0xff' already is in two bytes.

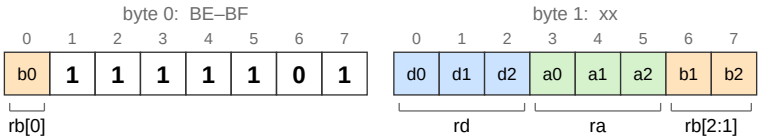
MUL — rd = low 16 bits of ra * rb; rd = low 16 bits of ra * immediate

Opcode	Instruction	Bytes	Cycles	Form	Operands
`BE–BF xx`	`mul rd, ra, rb`	2	2	three_reg	
`B8 xx`	`mul rd, rd, #imm5`	2	2	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
`B9 xx xx`	`mul rd, ra, #imm10`	3	2	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

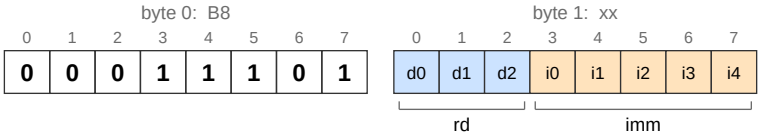
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

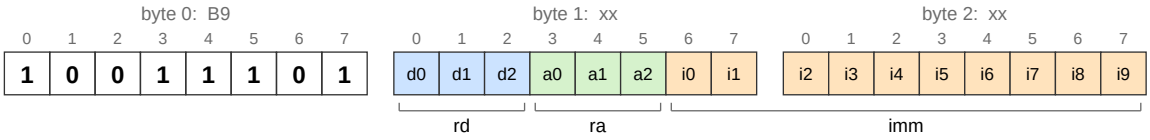
three_reg **`BE–BF xx`** `mul rd, ra, rb`



reg_imm5 **`B8 xx`** `mul rd, rd, #imm5`



reg_reg_imm10 **`B9 xx xx`** `mul rd, ra, #imm10`



Operation

mul rd, ra, rb

```
R[d] = R[a] * R[b];
```

Takes 1 cycle beyond the one an ALU instruction costs.

mul rd, ra, #imm

```
R[d] = R[a] * imm;
```

Takes 1 cycle beyond the one an ALU instruction costs.

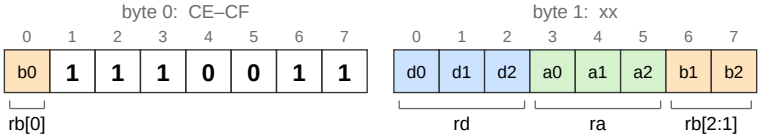
MULHS — rd = high 16 bits of ra * rb, signed

Opcode	Instruction	Bytes	Cycles	Form	Operands
`CE–CF xx`	`mulhs rd, ra, rb`	2	2	three_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **`CE–CF xx`** `mulhs rd, ra, rb`



Operation

```
R[d] = mulhs(R[a], R[b]);
```

Takes 1 cycle beyond the one an ALU instruction costs.

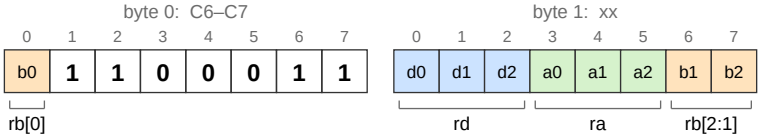
MULHU — rd = high 16 bits of ra * rb, unsigned

Opcode	Instruction	Bytes	Cycles	Form	Operands
C6–C7 xx	mulhu rd, ra, rb	2	2	three_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **C6–C7 xx** mulhu rd, ra, rb



Operation

```
R[d] = mulhu(R[a], R[b]);
```

Takes 1 cycle beyond the one an ALU instruction costs.

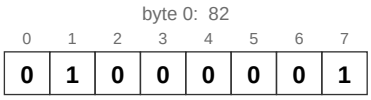
NOP — do nothing

Opcode	Instruction	Bytes	Cycles	Form	Operands
82	nop	1	1	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **82** nop



Operation

```
(nothing)
```

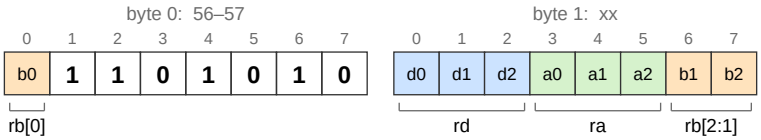
OR — rd = ra | rb; rd = ra | immediate

Opcode	Instruction	Bytes	Cycles	Form	Operands
`56–57 xx`	`or rd, ra, rb`	2	1	three_reg	
`84`	`or r0, r1, #0`	1	1	mov_r0_r1	All operands implicit
`85`	`or r1, r0, #0`	1	1	mov_r1_r0	All operands implicit
`86`	`or r2, r0, #0`	1	1	mov_r2_r0	All operands implicit
`50 xx`	`or rd, rd, #imm5`	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
`54 xx`	`or rd, rd, #immbit5`	2	1	reg_immbit5	immbit5: one of the 32 values below; ra is rd
`55 xx`	`or rd, rd, #immask5`	2	1	reg_immask5	immask5: one of the 32 values below; ra is rd
`52–53 xx`	`or rd, ra, #imm3`	2	1	reg_reg_imm3	imm3: one of the 8 values below
`51 xx xx`	`or rd, ra, #imm10`	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

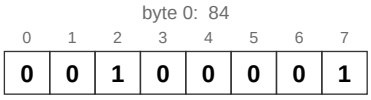
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **56–57 xx** `or rd, ra, rb`

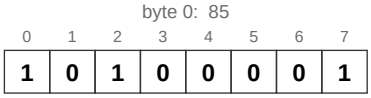


mov_r0_r1 **84** `or r0, r1, #0`

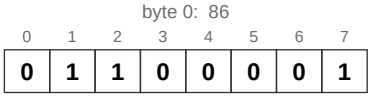


One-byte register moves. ‘or rd, ra, #0‘ is a copy, so pinning the two hot registers both ways gives single-byte ‘mov r0, r1‘ and ‘mov r1, r0‘ without a mov opcode existing at all. The ‘mov‘ alias rewrites to ‘or‘, and shortest-first selection lands on these.

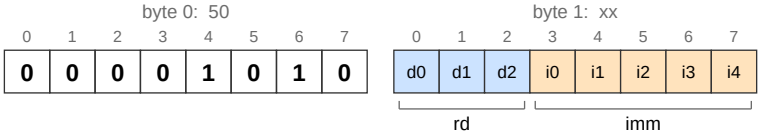
mov_r1_r0 **85** `or r1, r0, #0`



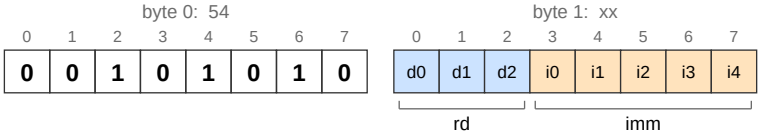
mov_r2_r0 **86** `or r2, r0, #0`



reg_imm5 **50 xx** `or rd, rd, #imm5`

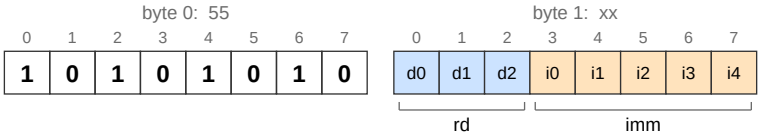


reg_immbit5 **54 xx** `or rd, rd, #immbit5`



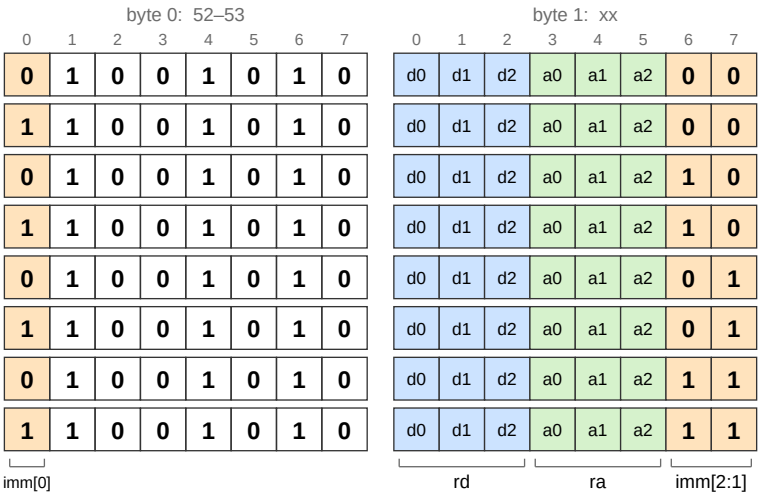
BIT SET. 'or rd, rd, #1<<n' for any n, in two bytes.

reg_immask5 **55 xx** `or rd, rd, #immask5`



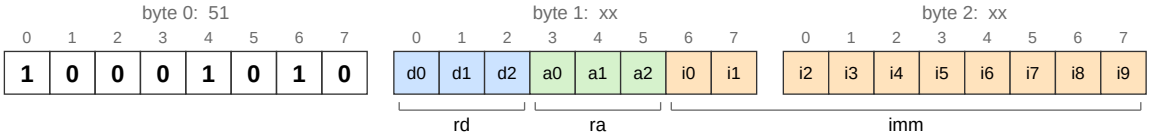
The same shape again, read through immask5: fields and stripes rather than single bits. Column .5 of the ALU block was reserved and empty.

reg_reg_imm3 **52–53 xx** `or rd, ra, #imm3`



52 00–3F `or rd, ra, #-1`
53 00–3F `or rd, ra, #0`
52 40–7F `or rd, ra, #1`
53 40–7F `or rd, ra, #2`
52 80–BF `or rd, ra, #3`
53 80–BF `or rd, ra, #4`
52 C0–FF `or rd, ra, #6`
53 C0–FF `or rd, ra, #8`

reg_reg_imm10 **51 xx xx** `or rd, ra, #imm10`



Operand Values

immbit5: value by field value

0	0x0001	1	0x0002	2	0x0004	3	0x0008	4	0x0010	5	0x0020	6	0x0040	7	0x0080
8	0x0100	9	0x0200	10	0x0400	11	0x0800	12	0x1000	13	0x2000	14	0x4000	15	0x8000
16	0xFFFE	17	0xFFFD	18	0xFFFB	19	0xFFF7	20	0xFFEF	21	0xFFDF	22	0xFFBF	23	0xFF7F
24	0xFEFF	25	0xFDFF	26	0xFBFF	27	0xF7FF	28	0xEFFF	29	0xDFFF	30	0xBFFF	31	0x7FFF

immask5: value by field value

0	0x000F	1	0x0003	2	0xFF00	3	0x000C	4	0x00F0	5	0x0030	6	0xF0F0	7	0x00C0
8	0x0F00	9	0x0300	10	0xCCCC	11	0x0C00	12	0xF000	13	0x3000	14	0xAAAA	15	0xC000
16	0xFFF0	17	0xFFFC	18	0x00FF	19	0xFFF3	20	0xFF0F	21	0xFFCF	22	0x0F0F	23	0xFF3F
24	0xF0FF	25	0xFCFF	26	0x3333	27	0xF3FF	28	0x0FFF	29	0xCFFF	30	0x5555	31	0x3FFF

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

or rd, ra, rb

```
  R[d] = R[a] | R[b];
```

or rd, ra, #imm

```
  R[d] = R[a] | imm;
```

Notes

or rd, ra, #0 is also written mov rd, ra; see MOV.

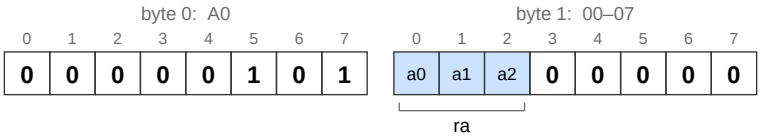
POP — pop a word into ra; pop two words, into ra then rb; pop three words, into ra then rb then rc

Opcode	Instruction	Bytes	Cycles	Form	Operands
A0 00–07	pop ra	2	3, odd 4	single	
0F	pop r2	1	3, odd 4	r2	All operands implicit
0D	pop r7	1	3, odd 4	lr	All operands implicit
A2 00–3F	pop ra, rb	2	4, odd 6	double	
A6–A7 xx	pop ra, rb, rc	2	5, odd 8	triple	

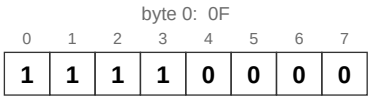
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

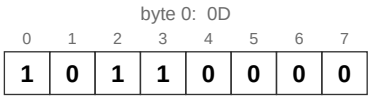
single **A0 00–07** `pop ra`



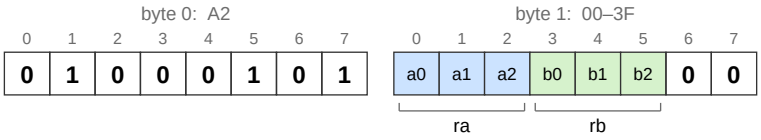
r2 **0F** `pop r2`



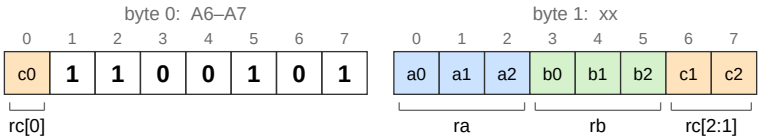
lr **0D** `pop r7`



double **A2 00–3F** `pop ra, rb`



triple **A6–A7 xx** `pop ra, rb, rc`



Operation

pop ra

```
base = sp;
R[a] = M16[base];
sp = base + 2;
```

pop ra, rb

```
base = sp;
R[a] = M16[base];
R[b] = M16[base + 2];
```

```
  sp = base + 4;
```

pop ra, rb, rc

```
  base = sp;
  R[a] = M16[base];
  R[b] = M16[base + 2];
  R[c] = M16[base + 4];
  sp = base + 6;
```

Notes

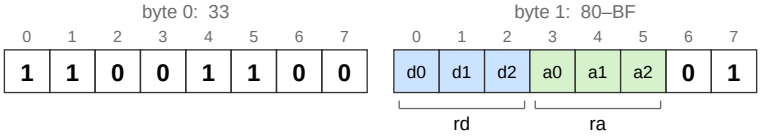
POPCOUNT — rd = number of set bits in ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
33 80–BF	popcount rd, ra	2	2	reg_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **33 80–BF** popcount rd, ra



Operation

```
R[d] = popcount(R[a]);
```

Takes 1 cycle beyond the one an ALU instruction costs.

Notes

Two cycles, like clz.

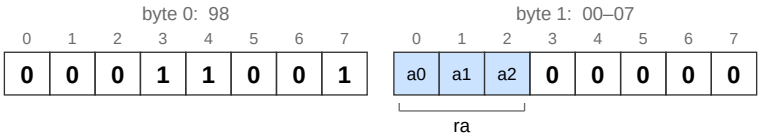
PUSH — push ra; push ra then rb; push ra, then rb, then rc

Opcode	Instruction	Bytes	Cycles	Form	Operands
`98 00–07`	`push ra`	2	3, odd 4	single	
`0E`	`push r0`	1	3, odd 4	r0	All operands implicit
`0C`	`push r7`	1	3, odd 4	lr	All operands implicit
`9A 00–3F`	`push ra, rb`	2	4, odd 6	double	
`9E–9F xx`	`push ra, rb, rc`	2	5, odd 8	triple	

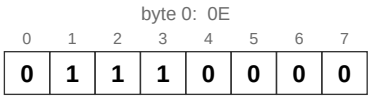
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

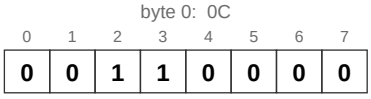
single **`98 00–07`** `push ra`



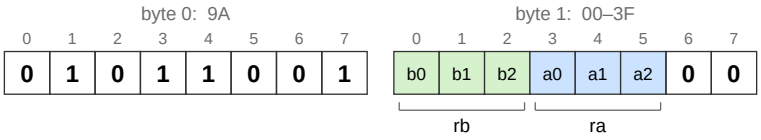
r0 **`0E`** `push r0`



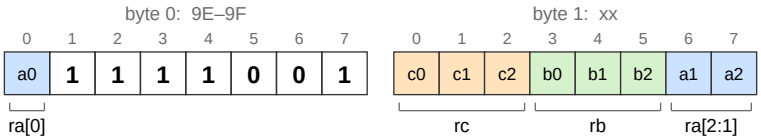
lr **`0C`** `push r7`



double **`9A 00–3F`** `push ra, rb`



triple **`9E–9F xx`** `push ra, rb, rc`



Operation

push ra

```
base = sp;
M16[base - 2] = R[a];
sp = base - 2;
```

push ra, rb

```
base = sp;
M16[base - 2] = R[a];
M16[base - 4] = R[b];
sp = base - 4;
```

push ra, rb, rc

```
base = sp;
M16[base - 2] = R[a];
M16[base - 4] = R[b];
M16[base - 6] = R[c];
sp = base - 6;
```

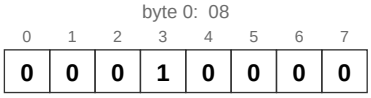
RET — pc = lr

Opcode	Instruction	Bytes	Cycles	Form	Operands
`08`	`ret`	1	2, wrong 5	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **`08`** `ret`



Operation

```
pc = lr;
```

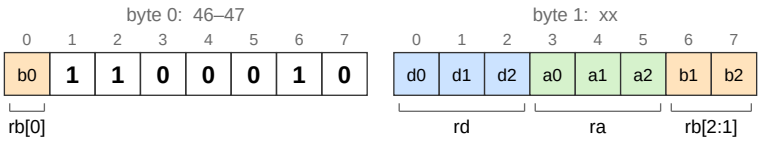
RSB — rd = rb - ra (reverse subtract); rd = immediate - ra

Opcode	Instruction	Bytes	Cycles	Form	Operands
`46–47 xx`	`rsb rd, ra, rb`	2	1	three_reg	
`40 xx`	`rsb rd, rd, #imm5`	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
`42–43 xx`	`rsb rd, ra, #imm3`	2	1	reg_reg_imm3	imm3: one of the 8 values below
`41 xx xx`	`rsb rd, ra, #imm10`	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

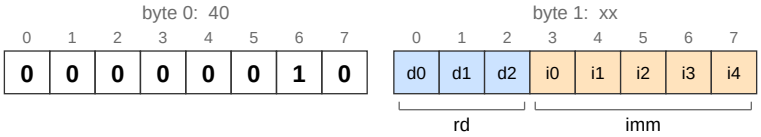
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

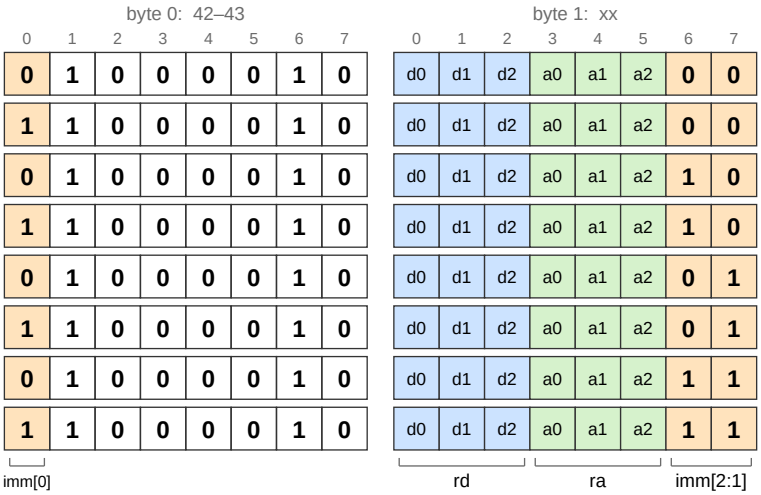
three_reg **46–47 xx** `rsb rd, ra, rb`



reg_imm5 **40 xx** `rsb rd, rd, #imm5`

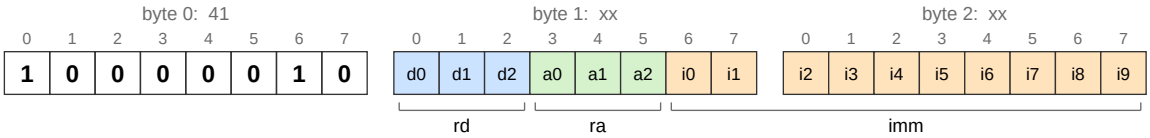


reg_reg_imm3 **42–43 xx** `rsb rd, ra, #imm3`



42 00–3F	`rsb rd, ra, #-1`	
43 00–3F	`rsb rd, ra, #0`	
42 40–7F	`rsb rd, ra, #1`	
43 40–7F	`rsb rd, ra, #2`	
42 80–BF	`rsb rd, ra, #3`	
43 80–BF	`rsb rd, ra, #4`	
42 C0–FF	`rsb rd, ra, #6`	
43 C0–FF	`rsb rd, ra, #8`	

reg_reg_imm10 **41 xx xx** `rsb rd, ra, #imm10`



Operand Values

imm3: value by field value

0	1	2	3	4	5	6	7
-1	0	1	2	3	4	6	8

Operation

rsb rd, ra, rb

```
R[d] = R[b] - R[a];
```

rsb rd, ra, #imm

```
R[d] = imm - R[a];
```

RTI — return from an interrupt

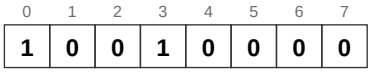
Opcode	Instruction	Bytes	Cycles	Form	Operands
09	rti	1	7	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **09** rti

byte 0: 09



Operation

```
pc = lr;
lr = shadow_lr;
shadow_isp = sp;
sp = shadow_sp;
ie = 1;
```

Notes

rti 1 byte

THE EXACT INVERSE OF THE ENTRY, statement for statement, which is what makes the pair safe to reason about. Note that it is not a swap in either direction: there are TWO shadow stack pointers, and each move is one way.

‘sp‘ AND ‘shadow_isp‘ ARE NEVER EXCHANGED IN ONE CYCLE, and that is the whole reason there are two shadows rather than one. A swap needs the old value of each side while the new one arrives, which on this machine means a mux in front of the register file’s write port - measured at 3.3 MHz, see the byte-serial core’s cpu.sv, and paid by every instruction rather than by the two that need it. Two one-way moves in two cycles cost the datapath nothing and the exception a cycle, and the exception is where a cycle is affordable.

IT IS ALSO THE BOOT SEQUENCE. With interrupts disabled at reset, the stub

```
mov lr, #user_entry_point
mov sp, #interrupt_stack_base
rti
```

loads the interrupt stack into ‘shadow_isp‘ on its way past and lands on user_entry_point with interrupts on - seven bytes, and both stack pointers are then set up. The garbage it leaves in sp is overwritten by the first thing user_entry_point does, and an interrupt arriving in that window is harmless because it swaps sp away before the handler can touch it.

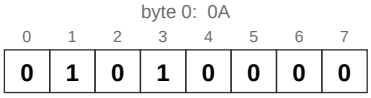
SEI — enable interrupts

Opcode	Instruction	Bytes	Cycles	Form	Operands
0A	sei	1	4	—	All operands implicit

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

— **0A** sei



Operation

```
ie = 1;
```

Notes

THE INTERRUPT-ENABLE FLAG IS WRITE-ONLY, and deliberately so. There is no instruction that reads it and no way to save and restore it, so a critical section is ‘cli‘ ... ‘sei‘ and the code is expected to know which state it was in. A readable flag invites the save-and-restore idiom, which needs a register to hold it and a conditional to act on it - and on a machine with eight registers that is a real cost paid at every critical section, to buy nesting that hand-written kernel code can simply not do.

THEY ARE ONE BYTE, AT 0x0a AND 0x0b, among the one-byte instructions that take more than a cycle. They used to be in the control row, at the price of making its length irregular, because the region was sixteen opcodes and worth more as abbreviations; at thirty-two it holds everything one byte long.

SHL — rd = ra << (rb & 15); rd = ra << (imm & 15)

Opcode	Instruction	Bytes	Cycles	Form	Operands
66–67 xx	shl rd, ra, rb	2	1	three_reg	
60 00–7F	shl rd, rd, #imm5	2	1	reg_imm5	imm5: 0 to 15; bit 4 ignored, written 0; ra is rd
8F	shl r0, r0, #1	1	1	r0_dbl	All operands implicit
62–63 xx	shl rd, ra, #shift3	2	1	reg_reg_imm3	shift3: one of the 8 values below
61 xx 00–03	shl rd, ra, #imm10	3	1	reg_reg_imm10	imm10: 0 to 15; bits 9:4 ignored, written 0

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **66–67 xx** `shl rd, ra, rb`

byte 0: 66–67

0	1	2	3	4	5	6	7
b0	1	1	0	0	1	1	0
rb[0]							

byte 1: xx

0	1	2	3	4	5	6	7
d0	d1	d2	a0	a1	a2	b1	b2
rd			ra			rb[2:1]	

reg_imm5 **60 00–7F** `shl rd, rd, #imm5`

byte 0: 60

0	1	2	3	4	5	6	7
0	0	0	0	0	1	1	0

byte 1: 00–7F

0	1	2	3	4	5	6	7
d0	d1	d2	i0	i1	i2	i3	0
rd			imm[3:0]				

r0_dbl **8F** `shl r0, r0, #1`

byte 0: 8F

0	1	2	3	4	5	6	7
1	1	1	1	0	0	0	1

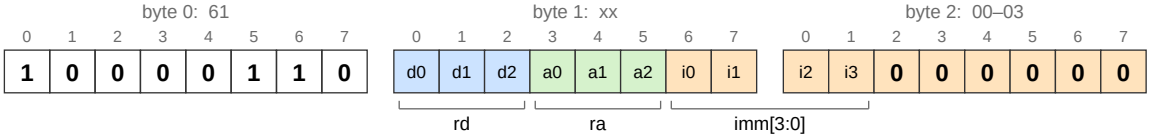
reg_reg_imm3 **62–63 xx** `shl rd, ra, #shift3`

byte 0: 62–63

byte 1: xx

byte 0								byte 1								
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	
0	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	0	0	**62 00–3F** `shl rd, ra, #15`
1	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	0	0	**63 00–3F** `shl rd, ra, #0`
0	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	1	0	**62 40–7F** `shl rd, ra, #1`
1	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	1	0	**63 40–7F** `shl rd, ra, #2`
0	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	0	1	**62 80–BF** `shl rd, ra, #3`
1	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	0	1	**63 80–BF** `shl rd, ra, #4`
0	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	1	1	**62 C0–FF** `shl rd, ra, #6`
1	1	0	0	0	1	1	0	d0	d1	d2	a0	a1	a2	1	1	**63 C0–FF** `shl rd, ra, #8`
imm[0]								rd			ra			imm[2:1]		

reg_reg_imm10 **61 xx 00–03** `shl rd, ra, #imm10`



Operand Values

shift3: value by field value

0 15 1 0 2 1 3 2 4 3 5 4 6 6 7 8

Operation

shl rd, ra, rb

```
R[d] = shl(R[a], R[b] & 15);
```

shl rd, ra, #imm

```
R[d] = shl(R[a], imm & 15);
```

Notes

Four forms each, six opcodes each: a tied register with a count, three registers, two registers with a table immediate, and two registers with any count. No one-byte encodings.

THE imm10 COLUMN IS FOR THE UNTIED PAIR, not for ten bits of count. The tied form at +0 has every count but only rd = ra, and the table at +2 has a free destination but only eight counts - so ‘lsr r1, r0, #13‘, which SHA-256’s rotations are made of, took a mov and a tied shift. +1 is the three-byte form that has both, and it costs the RTL nothing: the column’s immediate is imm10 for every row, and the shifter reads its low four bits.

Every shift uses only the low 4 bits of its count, whether that count comes from a register or an immediate. So a shift by 16 is a shift by 0, and a register count of -1 is a shift by 15. The ‘shift3‘ table spells that last one as 15 directly.

AN IMMEDIATE COUNT IN THE +0 AND +1 FORMS IS 0..15. There is never a reason to set the bits above those four, so the encodings that do are RESERVED - room for something later - and what they do is undefined. The hardware does not check: it reads the low four bits, so today a high bit changes nothing. The forms below still carry the whole imm5 and imm10, since the binutils generators take only fields of those shapes; until the assemblers learn the range, ‘shl r0, r0, #-1‘ assembles, and is a reserved encoding. The imm3 form keeps ‘shift3‘, whose -1 slot is what spells 15.

ST — store 16 bits; store 16 bits through a register index

Opcode	Instruction	Bytes	Cycles	Form	Operands
`12–13 xx`	`st rs, [ra, #imm3]`	2	3, odd 4	reg_reg	imm3: one of the 8 values below
`07`	`st r0, [r6, #0]`	1	3, odd 4	r0_sp	All operands implicit
`11 xx xx`	`st rs, [ra, #imm10]`	3	3, odd 4	reg_reg10	imm10: -512 to 511, sign-extended to 16 bits
`16–17 xx`	`st rs, [ra, rb]`	2	3, odd 4	reg_reg_reg	

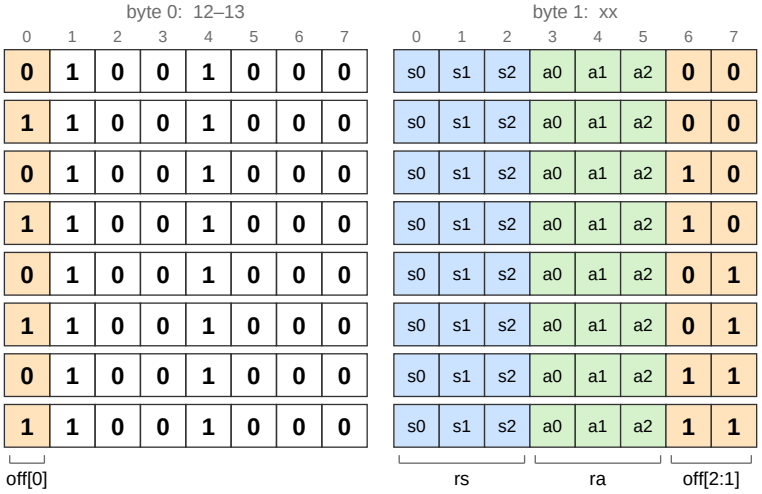
Assembler Aliases

Written	Assembles as	Description
`st rs, [ra]`	`st rs, [ra, #0]`	memory16[ra] = rs

Encodings

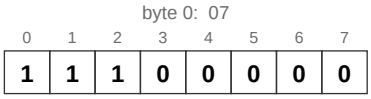
Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **12–13 xx** `st rs, [ra, #imm3]`

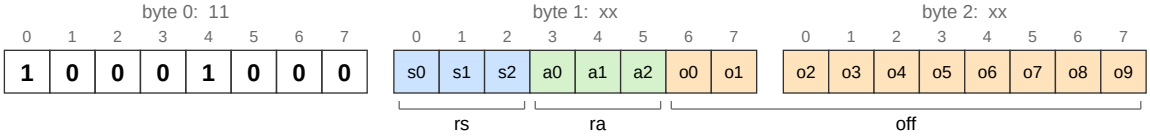


12 00–3F	`st rs, [ra, #-1]`	
13 00–3F	`st rs, [ra, #0]`	
12 40–7F	`st rs, [ra, #1]`	
13 40–7F	`st rs, [ra, #2]`	
12 80–BF	`st rs, [ra, #3]`	
13 80–BF	`st rs, [ra, #4]`	
12 C0–FF	`st rs, [ra, #6]`	
13 C0–FF	`st rs, [ra, #8]`	

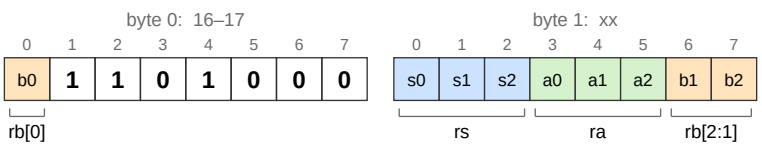
r0_sp **07** `st r0, [r6, #0]`



reg_reg10 **11 xx xx** `st rs, [ra, #imm10]`



reg_reg_reg **16–17 xx** `st rs, [ra, rb]`



Operand Values

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

st rs, [ra, #off]

```
M16[R[a] + off] = R[s];
```

st rs, [ra, rb]

```
M16[R[a] + R[b]] = R[s];
```

ST8 — store low 8 bits; store low 8 bits through a register index

Opcode	Instruction	Bytes	Cycles	Form	Operands
1A–1B xx	st8 rs, [ra, #imm3]	2	3	reg_reg	imm3: one of the 8 values below
19 xx xx	st8 rs, [ra, #imm10]	3	3	reg_reg10	imm10: -512 to 511, sign-extended to 16 bits
1E–1F xx	st8 rs, [ra, rb]	2	3	reg_reg_reg	

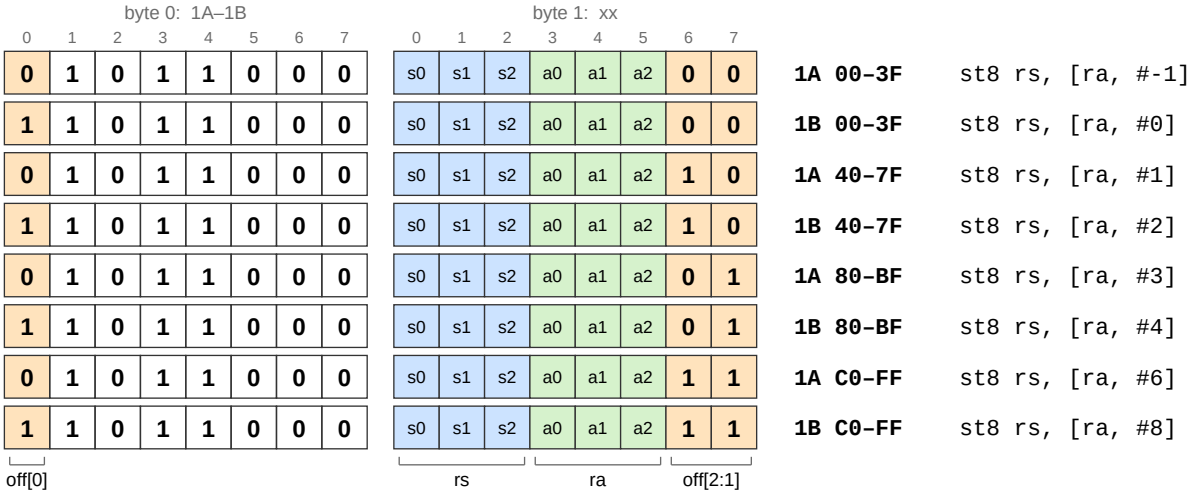
Assembler Aliases

Written	Assembles as	Description
st8 rs, [ra]	st8 rs, [ra, #0]	memory8[ra] = rs

Encodings

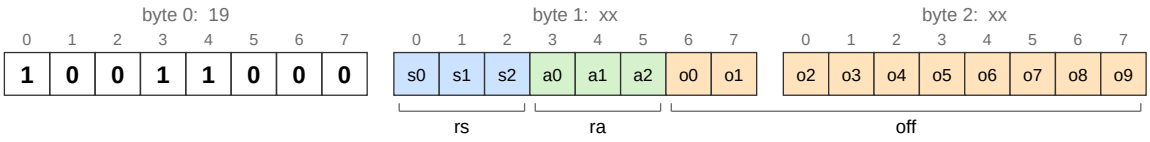
Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **1A–1B xx** st8 rs, [ra, #imm3]

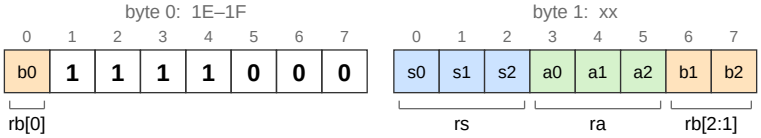


1A 00–3F	st8 rs, [ra, #-1]	
1B 00–3F	st8 rs, [ra, #0]	
1A 40–7F	st8 rs, [ra, #1]	
1B 40–7F	st8 rs, [ra, #2]	
1A 80–BF	st8 rs, [ra, #3]	
1B 80–BF	st8 rs, [ra, #4]	
1A C0–FF	st8 rs, [ra, #6]	
1B C0–FF	st8 rs, [ra, #8]	

reg_reg10 **19 xx xx** st8 rs, [ra, #imm10]



reg_reg_reg **1E–1F xx** st8 rs, [ra, rb]



Operand Values

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

st8 rs, [ra, #off]

```
M8[R[a] + off] = R[s];
```

st8 rs, [ra, rb]

```
M8[R[a] + R[b]] = R[s];
```

Notes

Stores get exactly two forms each - the two-register forms from the load block, and nothing else. There is no tied form (a store's source register is not usefully its own address register) and no implicit one-byte form (storing r0 through r0 at offset 0 is not a shape that occurs).

So a store is always at least two bytes, and its byte 1 is laid out exactly like a load's: register, address register, then the immediate remainder.

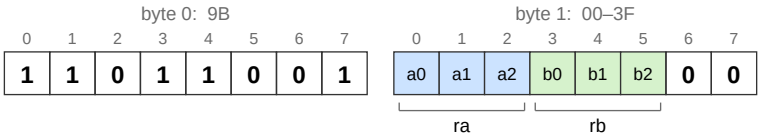
STM — store ra then rb through r5; store ra, then rb, then rc through r5

Opcode	Instruction	Bytes	Cycles	Form	Operands
9B 00–3F	stm ra, rb	2	4, odd 6	double	
9C–9D xx	stm ra, rb, rc	2	5, odd 8	triple	

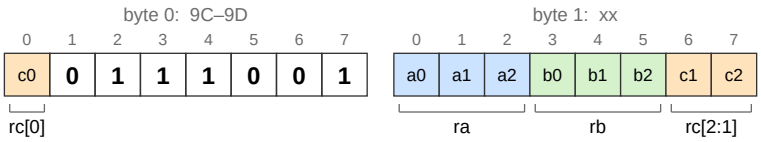
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

double **9B 00–3F** stm ra, rb



triple **9C–9D xx** stm ra, rb, rc



Operation

stm ra, rb

```
base = r5;
M16[base] = R[a];
M16[base + 2] = R[b];
r5 = base + 4;
```

stm ra, rb, rc

```
base = r5;
M16[base] = R[a];
M16[base + 2] = R[b];
M16[base + 4] = R[c];
r5 = base + 6;
```

Notes

push and pop walk the stack; these walk a buffer, and they are the same hardware doing it. ‘stm‘ stores through r5 and ‘ldm‘ loads through r1, each stepping its pointer past what it moved - so a copy loop is ‘ldm‘ then ‘stm‘ with no pointer arithmetic between them at all, and the two pointers advance themselves.

THE POINTER IS NAMED, NOT ENCODED, exactly as sp is for push and pop. That is what leaves all of byte 1 for the registers being moved, which is what makes a three-register form fit in two bytes. Two registers, so that a copy can hold both at once, and neither is sp.

r1 FOR THE LOAD because it is argument 1, which is the source in memcpy, memmove, strcmp and the rest: the routine reads through the register its caller already put the pointer in. r5 FOR THE STORE because it is the one register that is caller saved under every signature - see isa/abi.s - so a routine may walk it without saving it, and it never holds an argument that would have to be moved out of the way first. memcpy copies dest into r5 and leaves r0 alone as its return value; memset puts its fill word in r1, where c arrived. They were r1 for the store and r2 for the load until the RTL measured the change as free: the pointer is a register number in the predecode table, so a different one is a different constant, and nothing else in rtl/ moved.

THEY COUNT UP, where push counts down. A block move runs forwards, and the post-update leaves each pointer one past the last byte it moved - ready for the next iteration with nothing to add.

WHY THEY ARE WORTH SIX OPCODES. A memcpy built from push and pop reaches its speed by pointing sp at the buffers, and then spends more than a third of its cycles handing sp back and forth between the source and the destination. These have no handover at all: the pointers are different registers and each one updates itself.

THERE IS NO ONE-REGISTER FORM. There was, and it earned nothing: GCC never emits these, and every hand-written use

was the last word of a four-word move done as three then one, which costs the same as two then two - a load or a store is N + 2 cycles, so either split pays the overhead twice, and both are four bytes. A lone word is ‘ld‘ or ‘st‘, with an offset to reach it and an ‘add‘ only if the pointer is still wanted after. A "multiple" of one register was a misnomer anyway. Its slots, 0x99 and 0xa1 now, are free.

NOR IS THERE AN ldp, and it was tried: ‘ldp rd, rb, [ra]‘, any two registers from [ra] and [ra + 2] through any pointer, left as it was, in two opcodes, with a GCC peephole joining the split halves of a 32-bit load. It scored FructusMark -Os -mmul 127.5 -> 128.3 in the simulator, before any RTL, and a pointer taken from the instruction rather than the predecode table is a mux on the address path that could cost as much in MHz and cells. The post-increment kind - these two through any register - would have paid 0.2%: most adjacent pairs are 32-bit values read, changed and written back, so the pointer is still wanted. And the commonest of those, a counter’s x++, does better without one: GCC leaves its high word in memory unless the low word carries, 13 cycles to 8, which an ldp would only have made 11.

THE ORDER IS THE SAME AS push’s: the first register named is stored first, so with a pointer that counts up it lands at the LOWEST address. A save through stm and a reload through ldm therefore take the SAME register list, where a push/pop pair reverses it.

SUB — rd = ra - rb; rd = ra - immediate

An assembler alias: it has no encoding of its own.

Assembler Aliases

Written	Assembles as	Description
`sub rd, ra, rb`	`rsb rd, rb, ra`	rd = ra - rb
`sub rd, ra, #imm`	`add rd, ra, #-imm`	rd = ra - immediate

Encodings: see RSB.

Encodings: see ADD.

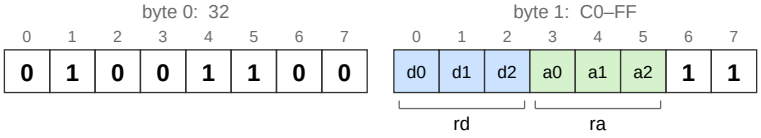
SXT8 — rd = ra sign extended from 8 bits

Opcode	Instruction	Bytes	Cycles	Form	Operands
32 C0–FF	sxt8 rd, ra	2	1	reg_reg	

Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

reg_reg **32 C0–FF** sxt8 rd, ra



Operation

```
R[d] = sxt8(R[a]);
```

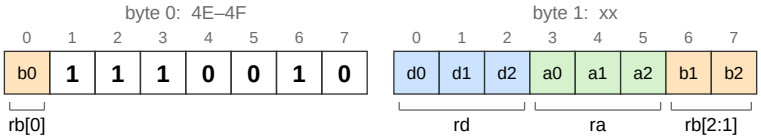
XOR — rd = ra ^ rb; rd = ra ^ immediate

Opcode	Instruction	Bytes	Cycles	Form	Operands
`4E–4F xx`	`xor rd, ra, rb`	2	1	three_reg	
`83`	`xor r0, r0, #1`	1	1	r0_not	All operands implicit
`48 xx`	`xor rd, rd, #imm5`	2	1	reg_imm5	imm5: -16 to 15, sign-extended to 16 bits; ra is rd
`4C xx`	`xor rd, rd, #immbit5`	2	1	reg_immbit5	immbit5: one of the 32 values below; ra is rd
`4D xx`	`xor rd, rd, #immask5`	2	1	reg_immask5	immask5: one of the 32 values below; ra is rd
`4A–4B xx`	`xor rd, ra, #imm3`	2	1	reg_reg_imm3	imm3: one of the 8 values below
`49 xx xx`	`xor rd, ra, #imm10`	3	1	reg_reg_imm10	imm10: -512 to 511, sign-extended to 16 bits

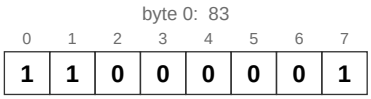
Encodings

Byte 0 is at the lowest address, and within each byte bit 0 is on the left, so an operand that runs on from one byte into the next reads left to right. Shaded bits are operand fields; a bracket names the operand and, where it is split, which of its bits it holds.

three_reg **4E–4F xx** `xor rd, ra, rb`



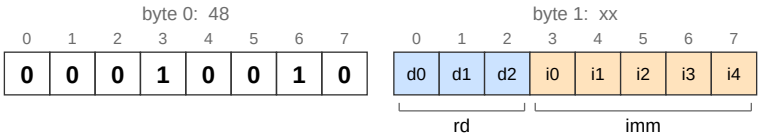
r0_not **83** `xor r0, r0, #1`



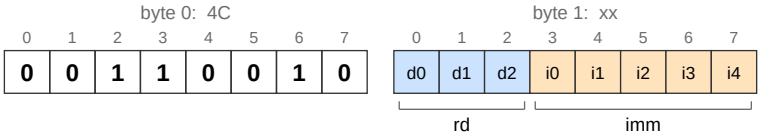
NEGATING A PREDICATE IS ONE BYTE. iseq and isset return 1 or 0 and never anything else, so ‘xor rd, rd, #1‘ is exactly ‘!‘ on their result - and r0 is where a return value lives, so a bool-returning function that wants ‘!=‘ where it computed ‘==‘ ends with this and a ‘ret‘, two bytes in total.

It costs one of the three free one-byte slots, and pins both operands, which is what an abbreviation is.

reg_imm5 **48 xx** `xor rd, rd, #imm5`



reg_immbit5 **4C xx** `xor rd, rd, #immbit5`

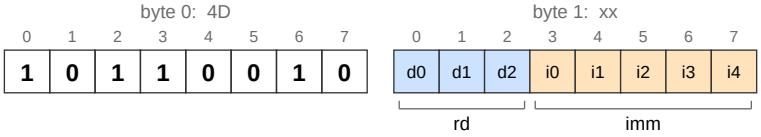


BIT FLIP, completing the set. ‘or #1<<n‘ sets a bit and ‘and #~(1<<n)‘ clears one; this toggles one, in the same two bytes and through the same 4-to-16 decoder. It costs an opcode the group had already reserved: slot +4 of every eight is the immbit5 slot, and it was free here only because nothing had asked for it yet.

The inverted half of the table is not wasted on xor either. ‘xor rd, rd, #~(1<<n)‘ flips every bit EXCEPT one, which is a complement with a single bit held back.

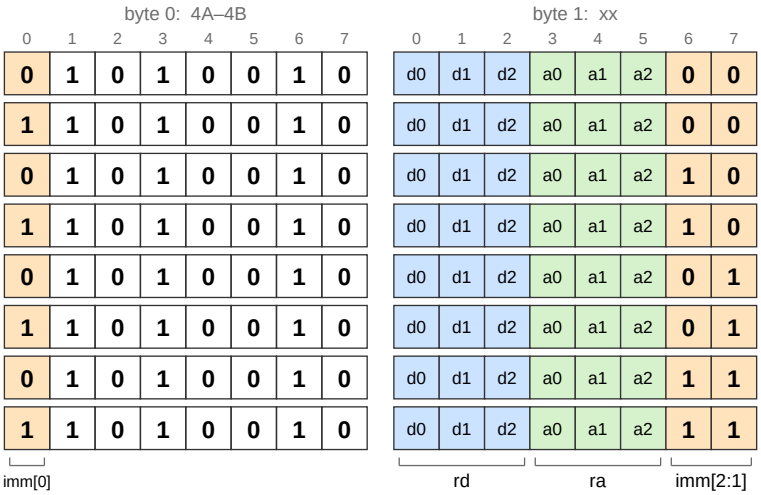
The PLAIN complement is not in this form and does not need to be: 0xffff is not ~(1<<n) for any n, but it is -1, which imm5 reaches - so ‘xor rd, rd, #-1‘ is already two bytes at 0x50. Where the two forms overlap the assembler prefers imm5, which is listed first, and drops the duplicate entries here.

reg_immask5 **4D xx** `xor rd, rd, #immask5`



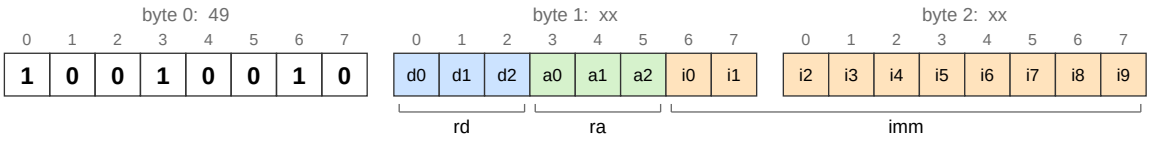
The same shape again, read through immask5: fields and stripes rather than single bits. Column .5 of the ALU block was reserved and empty.

reg_reg_imm3 **4A–4B xx** `xor rd, ra, #imm3`



4A 00–3F	`xor rd, ra, #-1`
4B 00–3F	`xor rd, ra, #0`
4A 40–7F	`xor rd, ra, #1`
4B 40–7F	`xor rd, ra, #2`
4A 80–BF	`xor rd, ra, #3`
4B 80–BF	`xor rd, ra, #4`
4A C0–FF	`xor rd, ra, #6`
4B C0–FF	`xor rd, ra, #8`

reg_reg_imm10 **49 xx xx** `xor rd, ra, #imm10`



Operand Values

immbit5: value by field value

0	0x0001	1	0x0002	2	0x0004	3	0x0008	4	0x0010	5	0x0020	6	0x0040	7	0x0080
8	0x0100	9	0x0200	10	0x0400	11	0x0800	12	0x1000	13	0x2000	14	0x4000	15	0x8000
16	0xFFFE	17	0xFFFD	18	0xFFFB	19	0xFFF7	20	0xFFEF	21	0xFFDF	22	0xFFBF	23	0xFF7F
24	0xFEFF	25	0xFDFF	26	0xFBFF	27	0xF7FF	28	0xEFFF	29	0xDFFF	30	0xBFFF	31	0x7FFF

immask5: value by field value

0	0x000F	1	0x0003	2	0xFF00	3	0x000C	4	0x00F0	5	0x0030	6	0xF0F0	7	0x00C0
8	0x0F00	9	0x0300	10	0xCCCC	11	0x0C00	12	0xF000	13	0x3000	14	0xAAAA	15	0xC000
16	0xFFF0	17	0xFFFC	18	0x00FF	19	0xFFF3	20	0xFF0F	21	0xFFCF	22	0x0F0F	23	0xFF3F
24	0xF0FF	25	0xFCFF	26	0x3333	27	0xF3FF	28	0x0FFF	29	0xCFFF	30	0x5555	31	0x3FFF

imm3: value by field value

0	-1	1	0	2	1	3	2	4	3	5	4	6	6	7	8

Operation

xor rd, ra, rb

```
R[d] = R[a] ^ R[b];
```

xor rd, ra, #imm

```
R[d] = R[a] ^ imm;
```